كلية الحوسبة والمعلوماتية |
College of Computing and Informatics

الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

College of Computing & Informatics (CCI)

SENIOR PROJECT-II REPORT

Bride Assistant App

Author(s):

200088111	Norah Nawaf Alqasem
210026034	Mudhi Khalaf Albuainain
210024612	Ragad Faisal Alqarni
210024963	Sarah Saad Aldawsari
200011269	Nouf Nasser Alalyani

Project Supervisor:

Dr. Huda Dakheel allah Aljohani

كلية الحوسبة والمعلوماتية |
College of Computing and Informatics

الجامعة السعودية الإلكترونية
SAUDI ELECTRONIC UNIVERSITY
2011-1432

Bride Assistant App

By:

Norah Nawaf Alqasem

Mudhi Khalaf Albuainain

Ragad Faisal Alqarni

Sarah Saad Aldawsari

Nouf Nasser Alalyani

Thesis/Project submitted to:

College of Computing & Informatics, Saudi Electronic University, Riyadh, Saudi Arabia.

In partial fulfillment of the requirements for the degree of:

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

Dr. Huda Dakheel allah Aljohani	
Project Supervisor	Project Committee Chair

ABSTRACT

Wedding planning can be a complex and overwhelming process, especially for brides in the Eastern Province of Saudi Arabia who may face unique cultural and logistical challenges. Traditional methods of wedding planning, often involving manual tasks and reliance on multiple sources of information, can be time-consuming, stressful, and prone to errors.

To address these challenges, the **Bride Assistant App** offers a comprehensive and personalized solution for brides in the Eastern Province. The app aims to streamline the wedding planning process by providing a centralized platform for managing various aspects of the event, from budgeting and venue selection to vendor coordination and guest list management and gift planning.

DEDICATION

This work is dedicated to our families for their unconditional support and love. Our professors, especially Dr. Huda Dakheel Allah Aljohani, whose guidance and expertise shaped the project's success. Lastly, the brides whose unique journeys inspired us to create this application. We hope it becomes a trusted companion that makes their special day unforgettable.

PREFACE

The **Bride Assistant App** project was inspired by the unique challenges that brides in the Eastern Province of Saudi Arabia face during wedding planning. Weddings are meant to be joyous celebrations filled with tradition, but they often come with a significant amount of logistical and emotional stress. This app was created to offer a practical solution that simplifies the planning process while honoring the cultural values and traditions that make weddings so special.

Working on this project has been a truly rewarding experience, and we could not have completed it without the support of many incredible people. We are deeply thankful to Dr. Huda Dakheel Allah Aljohani for her invaluable guidance and encouragement throughout this journey. Her mentorship provided clarity and direction at every step.

This project represents a collective effort to blend tradition and innovation. We hope that the **Bride Assistant App** becomes more than just a tool it should be a source of support and inspiration, making the process of wedding planning not only easier but also more enjoyable and memorable for brides.

REVISION HISTORY

Name	Date	Reason For Changes	Version
Nouf Alalyani	14 Apr	Added Conclusion and Future Work chapter 8	1.0
Mudhi Albuainain	16 Apr	Updated the future work on chapter 8 based on the supervisor request	2.0
Raghad Alqarni	16 Apr	Added the some code snippets based on the supervisor request	2.1
Norah Alqasem	16 Apr	Updated vendors and stores list based on the supervisor request	2.2
Sarah Aldawsari	20 Apr	Updated the Gantt chart by adding chapter 6, 7, and 8.	3.0
Norah Alqasem	20 Apr	Revised the Application interfaces description.	3.1
Mudhi Albuainain	20 Apr	Fixed grammatical errors and improved clarity in the file.	3.2
Nouf Alalyani & Raghad Alqarni	21 Apr	Updated the budget screen code by adding tracking expenses box based on user testing	4.0
Sarah Aldawsari & Mudhi Albuainain	22 Apr	Updated the wedding information screen code by adding a countdown box to the wedding day	4.1

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 10

1.1 Project Background/Overview: 10

1.2 Problem Description: 11

1.3 Project Scope: 11

1.4 Project Objectives: 12

1.5 Project Structure/Plan: 13

CHAPTER 2: LITERATURE REVIEW 16

CHAPTER 3: METHODOLOGY 24

CHAPTER 4: SYSTEM ANALYSIS 26

4.1 Product Features: ..... Error! Bookmark not defined.

4.2 Functional Requirements: ..... Error! Bookmark not defined.

4.3 Nonfunctional Requirements ..... Error! Bookmark not defined.

4.4 Analysis Models ..... Error! Bookmark not defined.

CHAPTER 5: SYSTEM DESIGN 36

CHAPTER 6: SYSTEM IMPLEMENTATION 41

CHAPTER 7: TESTING & EVALUATION 73

CHAPTER 8: CONCLUSION AND FUTURE WORK 85

6.1 Conclusion 85

6.2 Future Work 85

REFERENCES 86

APPENDIX: Glossary 89

TABLE OF TABLES

Table 1: Project Plan 13
Table 2: Comparison table of the studies 19
Table 3: User Registration 27
Table 4: User Login 28
Table 5: Manage Budget 29
Table 6: Vendor Directory 30
Table 7: Task Organization 31
Table 8: Guest List Management 32
Table 9: Wedding Information 33
Table 10: TC-User Create Account 73
Table 11: TC-User Login 74
Table 12: TC-Budget Management 75
Table 13: TC-Vendor Directory Access 76
Table 14: TC-Task Organization 77
Table 15: TC-Guest List Management 78
Table 16: TC-Add Wedding Information 79
Table 17: TC-Admin Manage Vendor Store 80
Table 18: TC-Forgot Password 81
Table 19: TC-User Logout 82
Table 20: TC-Add Comments and Rating 83
Table 21: TC-Social Media and Contact 84

TABLE OF FIGURES

Figure 1: Gant chart 14
Figure 2: Waterfall Methodology 25
Figure 3: use-case diagram 35
Figure 4: Entity relationship diagram 36
Figure 5: Complete class diagram 37
Figure 6: Sequence diagram 38
Figure 7: Component diagram 39
Figure 8: Deployment diagram 40
Figure 9: Admin login screen 42
Figure 10: Users list 42
Figure 11: Stores list 43
Figure 12: Adding store 43
Figure 13: Categories screen 44
Figure 14: Welcoming screen 45
Figure 15: Create account screen 46
Figure 16: Login screen 46
Figure 17: Stores screen 47
Figure 18: Detailed profile screen 48
Figure 19: Wedding information screen 49
Figure 20: Task screen 50
Figure 21: Menu screen 51
Figure 22: User profile screen 51
Figure 23: Guests list screen 52
Figure 24: Budget screen 53

CHAPTER 1: INTRODUCTION

1.1 Project Background/Overview:

Wedding planning is often regarded as one of the most significant and joyful milestones in a person's life. However, for many brides, especially in culturally rich regions like the Eastern Province of Saudi Arabia, this experience can quickly turn into a source of stress and confusion. The intricacies of organizing a wedding go beyond logistics; they involve navigating a complex landscape of cultural expectations, emotional investments, and financial considerations.

In the Eastern Province, weddings are steeped in cultural significance and tradition, making the planning process particularly detailed. Brides typically find themselves managing various responsibilities, such as:

- **Vendor Selection:** Finding reliable suppliers for entertainment, décor, catering, and photography can be challenging. With each vendor differing in availability, cost, and quality of service, making informed choices becomes a daunting task.
- **Budget Management:** Weddings can be costly, and without effective budgeting tools, brides may struggle to track expenses, risking overspending and financial stress.
- **Guest list management:** organizing a guest list, which often includes family, friends, and colleagues, involves not only tracking guests responses but also handling accommodations, dietary restrictions, and seating arrangements.
- **Task Organization:** Staying organized is crucial, as numerous tasks exist, from sending invitations to finalizing vendor contracts.
-

The emotional toll of wedding planning is significant. Brides often face heightened anxiety and stress, driven by the desire for perfection. Concerns about disappointing family members or failing to meet societal expectations only intensify these feelings. Traditional planning methods, which may involve scattered notes, spreadsheets, and multiple communication channels, often lead to a sense of chaos and overwhelm.

Current Solutions and Their Limitations

Although various wedding planning tools and resources are available, many do not address the specific needs of brides in the Eastern Province. Traditional approaches often rely on paper checklists, informal recommendations from friends, and haphazard online searches, resulting in inefficiencies and errors.

While some digital solutions exist, they frequently fall short in several key areas:

- **Lack of Local Focus:** Many apps do not accommodate local cultural practices or provide access to regional vendors, leaving brides without crucial resources.
- **User Experience:** Some applications are not user-friendly, making it difficult for users to navigate features or quickly find relevant information.
- **Limited Functionalities:** Many existing tools focus on specific aspects of wedding planning, such as budget tracking or vendor listings, but lack a comprehensive solution that integrates all necessary features into one platform.

Given the unique challenges faced by brides in the Eastern Province, there is a clear need for a dedicated application that offers a holistic approach to wedding planning. The Bride Assistant App aims to fill this gap by providing a centralized platform that simplifies the planning process while honoring local cultural nuances.

1.2 Problem Description:

Planning a wedding can be an overwhelming experience for brides. There are many details to consider while organizing a wedding, whereby brides often juggle multiple tasks, including vendor selection, budget tracking, and guest list management. Without a centralized tool, this can lead to confusion and oversight. In addition, the cost of weddings may add up, and keeping track of expenses can be very stressful. Many brides struggle with managing these complexities, leading to stress, missed deadlines, and a feeling of being unorganized.

1.3 Project Scope:

A user-friendly **Android** mobile app designed to simplify wedding planning for brides in the Eastern Province of Saudi Arabia. The **"Bride Assistant App"** is a comprehensive **Android** platform tailored to meet this need. It offers essential features such as budget management, a directory of local vendors, task organization capabilities, and reminder notifications. By connecting users with local businesses and organizing tools, the app aims to empower brides to take charge of their wedding planning, reduce feelings of being overwhelmed, and enhance overall experiences. Furthermore, it fosters a supportive community and contributes to the growth of the local economy.

The key stakeholders involved in this project include:

- **Brides-to-Be:** The app's main users who will derive benefits from its features.
- **Local Vendors:** Participants in the app's service directory (e.g., venues, beauty services).
- **Project Team:** The individuals who are accountable for the development of the application include developers, designers, and project administrators.
- **Investors/Sponsors:** Those who are providing financial support for the project.
- **Marketing Team:** The individual in charge of promoting the application to potential users.

The Proposed Solution:

The Bride Assistant App is an **Android** application designed to simplify wedding planning for brides in the Eastern Province of Saudi Arabia. The app addresses the unique challenges faced by brides-to-be by providing a user-friendly platform that combines essential planning tools with access to local resources.

1.4 Project Objectives:

1. **Facilitate the Wedding Planning Process**
   Make it easy and convenient for brides to plan every detail of their wedding with the help of accessible tools and features.

2. **Improve Budget Management**
   Make it easier for brides to keep tabs on their spending and stay within their budgets by adding an expense tracking function.

3. **Provide Venue and Service Booking Options**
   Make it simple to reserve wedding venues and halls using an intuitive interface that provides users with comprehensive details and evaluations to guide their decision-making process.

4. **Enhance Access to Beauty Services**
   Make it easy for people to get in touch with and book local hair and makeup artists by providing a list of their services and connecting to their social media accounts.

5. **Reminders and Task Organization**
   Make sure no critical thing is missed by providing brides with a reminder option to help them arrange assignments and distributions.

6. **Provide a Comprehensive Reference**
   For future reference, compile all pertinent links, images, and information regarding preparations in a reference section.

7. **Improve User Experience**
   Create a simple and easy-to-use interface to streamline the planning process for the bride.

8. **Raise Awareness of Local Services**
   Make it easy for brides in the Eastern Province to find available options by highlighting local service providers.

1.5 Project Structure/Plan:

1. Planning and research.
Describe the activities in the form of a table:

Table 1: Project plan.

No	Activity	Start	End	Duration of days
1	Introduction	Sep-05	Sep-20	16
2	Literature Review	Sep-20	Sep-27	8
3	Methodology	Sep-27	Oct-04	8
4	System Analysis	Oct-04	Oct-18	15
5	System Design	Oct-18	Oct-29	12
6	System Implementation	Dec- 21	Feb-28	70
7	Testing & Evaluation	Mar-2	Mar-27	26
8	Conclusion & Future Work	Apr- 13	Apr-19	7

Descriptive texts for each activity (see table 1**):**

- **Introduction:** This initial phase includes writing an introduction and planning the project, its purpose, and how it will be a suitable and helpful solution for brides. The bride's needs will be researched and the tools and services that can facilitate her wedding planning will be identified. Then the application's name will be chosen, and the project's scope will be determined.
- **Literature Review:** A search is conducted to find, evaluate, and analyze the work of others that can be compared to this project. The project objectives will be written in their correct perspective, and a proposed plan will be provided to accomplish them. Previous research will be evaluated to identify knowledge gaps and areas of application and development.
- **Methodology:** In this phase, the application is developed with continuous review and improvements. This methodology ensures the delivery of an application that meets the needs of users with the ability to continuously update and adapt to any changes.
- **System Analysis:** In this phase, all functional requirements will be expressed as use cases and include task and expense management, supplier list, and communication with them. The user can add and modify wedding-related tasks, track expenses, and receive and send to suppliers.
- **System Design:** The entire project plan will be included, covering relationships, sequence, user interface design, task management, budget, database, and each model will provide a guiding plan for developing the system.
- **System Implementation:** This phase focuses on the technical development and implementation of the app. It includes designing the user interface and user experience to ensure the app is user-friendly and culturally appropriate for brides in the Eastern

Province. The system structure will be determined, and the necessary technologies and development tools will be selected. During this phase, core features such as the supplier directory, budget tracking, task management, and guest list organization will be programmed and integrated. The app will then undergo internal testing to ensure that performance and usability meet project requirements.

- **Testing & Evaluation:** At this stage, the app will be thoroughly tested to detect and fix any bugs or performance issues. Various types of testing will be conducted, such as functionality testing, usability testing, and cross-device compatibility testing. Feedback will also be collected from a group of targeted brides in the Eastern Province to assess the app's effectiveness, ease of use, and cultural appropriateness. Based on the results, necessary improvements will be made to enhance the app's overall quality and reliability.

- **Conclusion & Future Work:** In this final phase, previous stages of the project will be reviewed, highlighting potential improvements that could contribute to the project's development and quality. Future ideas and proposals will also be considered for implementation to ensure the app's continued development and effective continuity in meeting user needs.

- **Gantt chart:**

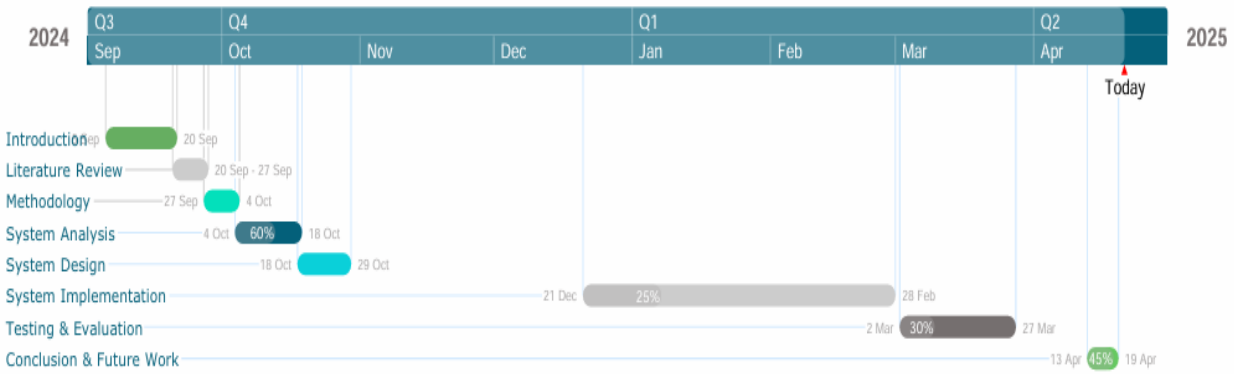

Figure 1: Gant chart.

This chart displays the timeline for the project tasks from September to April 2025, including activities such as the introduction, literature review, methodology, system analysis, system design, and discussion & conclusion.

CHAPTER 2: LITERATURE REVIEW

Many studies have been found in this field and they will be listed below:

Sangchan (n.d.) examines the wedding planning and management process in *Wedding Event Planning and Management: A Case Study of The Hug Wedding Planner and Decor*. This study covers pre-planning, design, supplier coordination, and event execution to create unforgettable experiences for high-end clients. Utilizing interviews and observations from an internship, it highlights the complexities of the wedding industry, the importance of client collaboration, and the meticulous planning needed to realize their visions.

Similarly, Senarath and Rajarata University of Sri Lanka (2021) discuss three software development methodologies in *Waterfall Methodology, Prototyping, and Agile Development*. The Waterfall model offers a linear process that is easy to manage but inflexible, while Prototyping allows for user feedback, enhancing involvement but risking scope creep. Agile, on the other hand, emphasizes iterative development with frequent customer feedback, improving responsiveness but requiring a cultural shift.

Additionally, Durgekar et al. (2024) highlight the importance of User Interface (UI) and User Experience (UX) in *A Review Paper on the Design and Experience of Mobile Applications*. They note that many apps fail to engage users due to poor design and stress the need for visually appealing, intuitive interfaces that consider user needs. The paper emphasizes that effectively integrating UI and UX is crucial for developing successful mobile applications that satisfy users.

In addition, the document *User-Review Oriented Social Recommender System for Event Planning* focuses on wedding planning and introduces a hybrid framework combining content-based, collaborative, demographic, and knowledge-based filtering. This system generates personalized vendor suggestions based on users' social network interactions. A study with Facebook users demonstrated that the framework effectively improves user satisfaction by providing relevant recommendations tailored to individual preferences and locations.

Furthermore, *Saudi Weddings: A Traditional Affair* (n.d.) explores traditional practices in Saudi weddings, including the proposal process, dowry negotiations, and the role of family. It also highlights the influence of modern adaptations, such as the use of digital tools and social media, on contemporary bridal experiences.

The study *Design Elements on Wedding Planner Platforms as the Fundamental of User Interface Design* emphasizes the importance of user interface design for enhancing experiences on wedding planning platforms. It explores key elements like input controls and navigation, highlighting principles such as familiarity and consistency. Moreover, the findings identify ten essential design elements, underscoring the need for a consistent and user-friendly design process.

Meanwhile, *Wedding Planner For Dummies* serves as a helpful guide for couples planning their weddings. It covers everything from budgeting and choosing a venue to coordinating vendors and creating timelines. Packed with practical tips and handy checklists, it simplifies the planning process and provides strategies to help couples personalize their special day. This resource is designed to be easy to use, making it ideal for anyone navigating the complexities of wedding planning.

Moreover, the article *Embracing the Evolution of Weddings* (Wedding Academy Global, 2024) looks at exciting trends transforming the wedding industry today. It highlights how digital tools and sustainability practices are reshaping the planning process. Notable innovations include virtual reality for venue tours and AI for personalizing wedding details, along with a growing focus on eco-friendly celebrations. The article underscores how these advancements blend traditional elements with modern ideas, making weddings more inclusive and personalized.

Additionally, the article *Marketing Strategies for the Wedding Industry* (Ecojoin Journal) examines how digital tools and marketing techniques are revolutionizing the wedding industry. It highlights strategies like social media marketing, search engine optimization (SEO), and content-driven campaigns that help wedding businesses increase their visibility and engage with clients. The study also explores how these strategies impact customer behavior, vendor success, and the overall growth of the industry.

Finally, the report *Saudi Arabia Wedding Services Market* (MarkWide Research) analyzes the trends shaping the wedding services industry in Saudi Arabia. It discusses how cultural traditions influence the market, the growing demand for luxury services, and the integration of modern technologies like digital planning tools and social media marketing. Additionally, it highlights the key drivers, challenges, and opportunities for future growth in the market.

In another context, Lamothe, Guéhéneuc, and Shang offer a thorough review of the literature on API evolution. They emphasize how crucial APIs have become in software development, particularly with the rise of application packages. However, they note the challenges in developing and using APIs, indicating that while some issues have been addressed, many still require attention.

On the other hand, Garcia-Revilla et al. (2023) conducted a review of mobile applications for event management, focusing on those available for iOS devices. Their analysis highlighted the significant impact of technology on event organization, noting that specialized event management applications offer a wide range of advantages to facilitate planning and management. The study also emphasized the educational potential of these applications, suggesting that they can enhance learning and skill acquisition in event management education.

Meanwhile, Vyas (2023) addressed security challenges in Java application development, identifying common vulnerabilities such as SQL injection, cross-site scripting (XSS), and insecure dependencies. The paper proposed solutions, including secure coding practices, regular dependency audits, and robust authentication mechanisms to mitigate these risks. These insights are crucial for developers aiming to build secure wedding planning applications, ensuring the protection of user data and maintaining application integrity.

Additionally, Chougale et al. (2021) explored the use of Firebase, a cloud-based platform, in application development. They discussed various services offered by Firebase, such as real-time database management, authentication, cloud messaging, and performance monitoring. The paper highlighted Firebase's advantages in managing large amounts of random data efficiently, which is beneficial for applications requiring real-time data synchronization and user authentication. Implementing Firebase in wedding planning applications can enhance data management and provide a seamless user experience.

Furthermore, Pierce and Jiang (2021) explored how cultural values impact mobile marketing acceptance in the U.S. and China. They found that individual cultural traits, such as uncertainty avoidance and long-term orientation, significantly influence attitudes toward mobile marketing. Additionally, factors like risk avoidance and the need for permission were pivotal in shaping user perceptions in both countries. These insights underscore the importance of considering cultural nuances when developing wedding planning applications to ensure they resonate with diverse user bases.

Finally, in the realm of mobile application development, energy efficiency is a critical consideration. The development of tools like EcoAndroid, an Android Studio plugin, exemplifies efforts to enhance energy efficiency in mobile apps. EcoAndroid automatically applies energy-saving patterns to Java source code, thereby optimizing the energy consumption of applications. Implementing such tools in the development of wedding planning applications can lead to more sustainable and user-friendly products.

Table 2: Comparison table of the studies.

Comparison table of studies related to wedding planning, outlining their advantages and disadvantages.

Studie	Main idea	Advantages	Disadvantages
Wedding Event Planning and Management: A Case Study of The Hug Wedding Planner and Decor (Thammasat University)	The study explores wedding management through a case study of The Hug Wedding Planner and Decor.	• Highlights client centered approaches. • Highlights innovative design elements. • Showcases efficient supplier relationships.	• Focus on a single case study. • Undisclosed proprietary methods and financial details. • Heavy reliance on time-sensitive tasks.
Waterfall, Prototyping, and Agile Development (Rajarata University of Sri Lanka, 2021)	The study reviews software development methodologies: Waterfall, Prototyping, and Agile.	• **Waterfall:** Simple, clear documentation, good oversight. • **Prototyping:** User involvement, early defect detection, clear requirements. • **Agile:** Responsive to change; fosters collaboration; regular delivery.	• **Waterfall:** Inflexible; late testing delays feedback and poses high risks. • **Prototyping:** Risks inadequate analysis, scope creep, and high costs. • **Agile:** Requires organizational change for better customer involvement.
Design and Experience of Mobile Applications (ICST Transactions on Scalable Information Systems, 2024)	The study reviews the importance of UI and UX in mobile app design.	• Enhance user engagement and retention. • Improved user satisfaction. • Attractive design. • Clear navigation. • Focus on user needs and expectations	• Resource-intensive in terms of time and cost. • Risk of neglecting important features while focusing on design. • Challenges in keeping up with evolving design trends.
User-Review Oriented Social Recommender System for Event Planning.	The study proposes a hybrid recommender system for personalized wedding vendor suggestions based on social network interactions.	• Tailored recommendations • Multiple filtering techniques • Increased user satisfaction	• Requires user data • Cold start issue • Wedding-Focused
Saudi Weddings: A Traditional Affair.	The article examines traditional Saudi weddings and their modern adaptations.	• Detailed insights into traditions. • Highlights technology's role. • Engaging for diverse readers. • Balances history and modern trends.	• Lacks academic rigor. • No cultural comparisons. • Superficial modern insights. • No citations, raising reliability concerns.
Design Elements on Wedding Planner Platforms as the Fundamental of User Interface Design.	The study highlights how user interface design improves user experience on wedding planning platforms.	• Enhances user experience • Provides practical design guidance. • Emphasizes key principles.	• Limited to specific applications. • Needs more testing. • Can complicate design integration.
Woolsey, D. (2020). Wedding Planner For Dummies. Wiley.	The book provides couples with a practical guide to efficiently	• Comprehensive coverage.	• Limited depth on niche themes. • Lacks personalized advice.

	organize their weddings.	• Simplifies planning with tips and checklists. • Easy to understand.	• Information may become outdated.
Embracing the Evolution of Weddings.	The article examines how technological innovations and sustainability trends are transforming the wedding industry.	• Simplifies planning. • Promotes eco-friendliness. • Combines tradition with innovation.	• Inaccessible tech for some. • May reduce personal touches. • Costs limit sustainability practices.
Marketing Strategies for the Wedding Industry.	The article discusses how digital tools and marketing strategies are transforming the wedding industry.	• Increases vendor reach. • Strengthens client relationships. • Cost-effective for all business sizes.	• Vulnerable to platform changes. • Needs ongoing effort. • Requires careful data handling.
Saudi Arabia Wedding Services Market.	The report looks at how traditions, luxury demand, and technology shape Saudi Arabia's wedding market.	• Offers trend insights. • Highlights tradition and modernity. • Identifies growth opportunities.	• May overlook regional differences. • Could favor vendors. • Data access may require purchase.
A Systematic Review of API Evolution Literature.	The study reviews challenges and trends in the evolution of application programming interfaces (APIs) in software development.	• Thorough summary of research. • Highlights key API challenges. • Analyzes publication trends.	• More focus on challenges than solutions. • May miss some relevant studies. • Conclusions could be too broad.
User Acceptance of Mobile Event Apps: A Cross-Cultural Study	This study investigates the factors that influence the acceptance and usage of mobile event apps across different cultures. It likely examines cultural values, social norms, and individual preferences that impact user attitudes and behaviors towards these apps.	• Analyzes user experience (UX) factors influencing mobile event app acceptance. • Provides insights into user preferences and expectations for mobile event management tools.	• Focuses on general event apps, not specifically wedding planning. • Study conducted in 2021, might not reflect the latest trends.
Advances in event management using new technologies and mobile applications	This research explores the significant role of technology, particularly mobile applications, in transforming the event management industry. It likely discusses how mobile apps are being used for various aspects of event planning.	• Highlights the benefits of mobile applications in event planning. • Discusses the impact of technology on event organization and optimization.	• Lacks specific focus on wedding planning applications. • Doesn't delve into technical details of mobile app development.
Security Challenges and Solutions in Java	This study identifies and addresses security vulnerabilities in Java	•Identifies common security vulnerabilities in Java applications.	• Not directly related to wedding planning app functionalities.

Application Development	applications, covering threats like data breaches and unauthorized access, and discussing mitigation strategies such as secure coding, data encryption, and vulnerability testing.	• Offers solutions and best practices for secure coding.	• Focuses on Java development, might not apply to other programming languages.
A Framework for Intelligent Wedding Planning with User Preferences and Constraints	This research proposes a framework for intelligent wedding planning systems that personalize the experience based on user preferences and constraints, utilizing AI and machine learning techniques.	• Proposes a framework for intelligent wedding planning systems. • Discusses incorporating user preferences and constraints into the planning process.	• Might not provide specific implementation details.
Firebase - Overview and Usage	This document reviews Firebase, a Google-backed platform for mobile app development, covering its services—real-time databases, authentication, cloud messaging, and analytics—and their benefits for developers.	• Provides a general introduction to Firebase, a cloud platform for mobile app development.	• Lacks in-depth information on specific functionalities.

As shown in Table 2, the comparison of various studies highlights their main ideas, advantages, and disadvantages. Each study contributes unique insights, ranging from innovative wedding planning methods and user-focused design principles to software development approaches and digital marketing strategies. While some focus on practical applications, like Wedding Planner For Dummies or User-Review Oriented Social Recommender System for Event Planning, others explore broader trends, such as Embracing the Evolution of Weddings and Saudi Arabia Wedding Services Market. However, limitations like narrow scopes, resource demands, or lack of rigorous testing emphasize the need for further research and refinement to address gaps and expand their applicability.

That was for the studies, now the related applications:

Mila Celebrations, Wedding Planner, The Big Day, Honeyfund Wedding Registry, Dream Wedding, Honeydue, WeddingHappy, The Schedule Planner, and Save the Date - RSVP Your Event each offers distinct solutions for couples navigating wedding planning.

Mila Celebrations simplifies event planning with tools for managing guest lists, tracking responses, and organizing schedules. Its user-friendly interface enhances convenience, though limited customization options may overwhelm some users.

Wedding Planner serves as an all-in-one platform, combining budgeting, scheduling, and guest management. While its extensive features and customization options are beneficial, the complexity can be daunting, and technical issues may disrupt the experience.

The Big Day focuses on creating a countdown experience alongside budget management and vendor tracking. Its countdown feature adds a fun element, but it lacks advanced functionalities and has been criticized for an outdated interface.

Honeyfund Wedding Registry provides a unique cash registry for experiences or honeymoon contributions, appealing to couples but potentially confusing guests unfamiliar with cash registries. Couples may also have less control over how funds are used.

Dream Wedding, a mobile app by AbuOdeh and Atabeh, connects users with service providers like venues and photographers, offering booking and chat functions. While it enhances user experience and supports local businesses, it requires constant internet access and has a learning curve for developers. Plans include adding online payment options.

Honeydue is a free app for managing finances, enabling couples to track bills and budgets collaboratively. While it includes features like transaction categorization and alerts, its US-centric design might limit its usefulness internationally, and both partners need to use it regularly for effectiveness.

WeddingHappy features a customizable task calendar that adjusts to the wedding date, helping couples stay organized. However, its free version has limited features and advanced options require in-app purchases, which may deter those seeking a more interactive experience.

The Schedule Planner, To-Do List, and Reminders app helps manage time by allowing users to create to-do lists and set reminders. While user-friendly, it has ads and lacks advanced features like schedule sharing.

Finally, **"Save the Date - RSVP Your Event"** simplifies event planning and guest management by enabling users to send invitations and track responses. However, its focus on RSVP functions means it lacks broader wedding planning tools, and some features may require in-app purchases, making it more suitable for tech-savvy users.

These tools reflect a variety of approaches to wedding planning, highlighting the importance of user experience, flexibility, and collaboration in creating memorable events.

Overall, **The Bride Assistant App** stands out as a culturally tailored, all-in-one wedding planning solution designed specifically for brides in the Eastern Province of Saudi Arabia. Unlike other

apps that provide fragmented services, it integrates budget management, vendor coordination, guest list tracking, and task organization into a single, user-friendly platform.

What sets it apart is its focus on local traditions, direct access to Saudi vendors, and an intuitive, stress-free experience. Unlike the other apps, it ensures personalized planning tools that align with cultural values, making the wedding journey more seamless and enjoyable. With its comprehensive features and future expansion potential, the Bride Assistant App redefines wedding planning by making it efficient, organized, and deeply connected to traditions.

CHAPTER 3: METHODOLOGY

This section details the approach selected for the **Bride Assistant App** project, focusing specifically on the software development process. The Waterfall methodology has been adopted for its organized and sequential framework, which is especially advantageous when project requirements are well-defined and stable.

3.1 Reasons why waterfall methodology has been chosen

The decision to utilize the Waterfall methodology is based on several important factors First, the Waterfall model offers a clear and systematic approach to project management, facilitating organized progression through distinct phases. Each phase has specific deliverables, which simplifies progress tracking and timeline management. Second The linear progression of the Waterfall model ensures that each stage of requirements gathering, system design, implementation, testing, deployment, and maintenance is completed in order. This structure minimizes ambiguity and allows for comprehensive documentation at each step, which is essential for maintaining clarity throughout the development process. Third A strong emphasis on documentation ensures that all requirements, designs, and modifications are thoroughly recorded. This is especially useful for future updates and ongoing maintenance, as it enables developers to reference past decisions and understand the reasoning behind design choices. Forth Because the primary users of the Bride Assistant App are brides in the Eastern Province of Saudi Arabia, the initial requirements are anticipated to remain relatively stable. The Waterfall methodology is well-suited for projects where the scope is clearly defined from the beginning, reducing the likelihood of iterative changes.

3.2 Software Development Process

The development process for the **Bride Assistant App** utilizing the waterfall technique comprises the following main stages:

- **Requirements Gathering:** Initial meetings will be made with stakeholders, including brides-to-be and local vendors, to gather and record critical needs. This phase is intended to reflect the individual demands and obstacles faced throughout wedding preparation.
- **System Design:** Detailed design papers will be generated that outline the system architecture, database design, and mockups of the user interface. This phase will also include tactics for providing an intuitive user experience.
- **Implementation:** The application will be created in accordance with the requirements provided in the design papers, using a linear approach to ensure adherence to the planned design.
- **Testing:** Following installation, a complete testing phase will be conducted, including unit testing, integration testing, and user acceptability testing (UAT), to ensure that the application satisfies the defined requirements and performs properly.
- **Maintenance:** After deployment, the program will enter a maintenance phase, during which it will provide continuing support, updates, and problem fixes as needed. User comments gathered during this phase will help to shape future upgrades.

Using the waterfall technique, the Bride Assistant App seeks to provide a holistic solution that efficiently fulfills brides' demands during the wedding planning process, resulting in a more organized and joyful experience.

Waterfall Methodology:

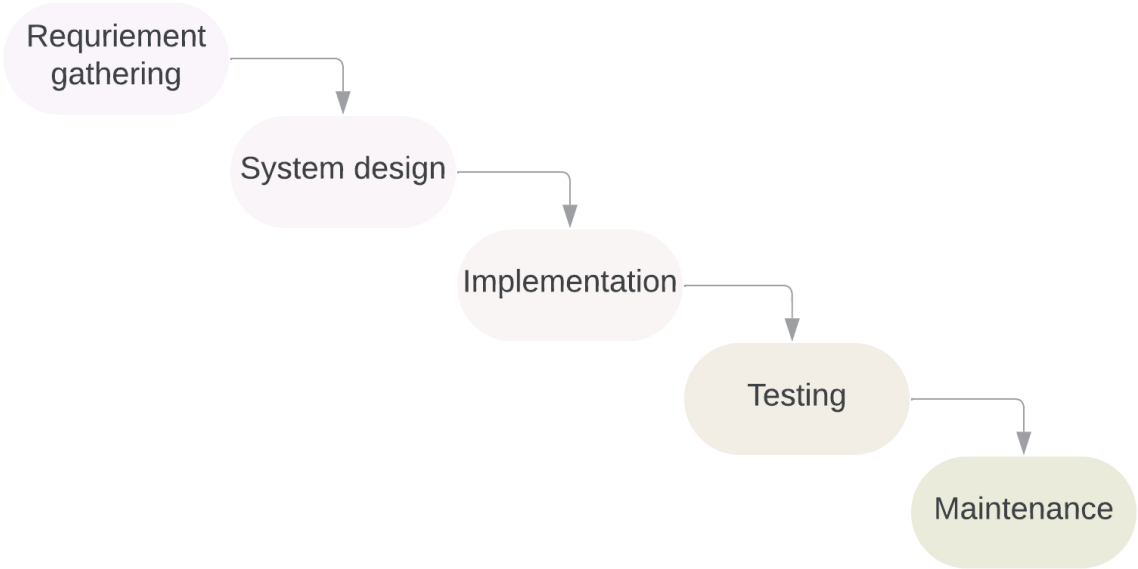

Figure 2: Waterfall methodology.

This diagram represents the sequential phases of the Waterfall methodology, including requirement gathering, system design, implementation, testing, and maintenance.

3.3 Tools and Techniques

To support the Waterfall development process, the following tools and technologies will be utilized:

- **Project Management:** Tools like Microsoft Project or Gantt charts will be used to manage timelines, track progress, and facilitate team communication.
- **Version Control:** Git will be employed for version control, ensuring that all code changes are tracked and managed effectively.
- **Development Framework:** The app will be developed using Java for Android, leveraging frameworks that enhance user experience and performance

CHAPTER 4: SYSTEM ANALYSIS

4.1 Product Features:

The **Bride Assistant App** is designed to provide a variety of essential features to aid brides in the Eastern Province of Saudi Arabia. Each part is made to help brides with a certain aspect of wedding planning so that they may relax and enjoy the process. The following is an in-depth review of the primary product features:

1. **User Authentication and Account Management**
   - Create and manage user accounts
   - Secure login and password recovery

2. **Budget Management**
   - Set a budget and track expenses
   - Generate visual reports
   - Receive budget alerts

3. **Vendor Directory**
   - Search for vendors by category
   - View vendor profiles and reviews
   - Contact vendors directly

4. **Task Management**
   - Create and manage to-do lists

5. **Guest List Management**
   - Create and manage guest lists

6. **Wedding Information**
   - View the countdown to the wedding day.

7. **User-Friendly Interface**
   - Intuitive navigation and customizable themes

4.2 Functional Requirements:

Use-Case 1: User Registration

Table 3: User registration.

Identifier	UC-UserRegistration	
Purpose	To enable new users to create an account.	
Priority	High	
Pre-conditions	The user is signing up.	
Post-conditions	The account has been created and successfully verified.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	The user fills out the registration form	After receiving the input, the system verifies it.
2	Registration form is submitted	A confirmation email is sent by the system.
3	User is redirected to the dashboard	The user can start using the app.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	The user submits incomplete data	Missing information is requested by the system.
2	Registration is canceled	The user is taken back to the login screen.

This use case outlines the steps for users to log in, detailing both the typical actions taken and the responses for alternate scenarios, as shown in table 3.

Use-Case 2: User Login

Table 4: User login.

Identifier	UC-UserLogin	
Purpose	To allow users to access their accounts.	
Priority	High	
Pre-conditions	The user's account is active and verified.	
Post-conditions	The app user has successfully logged in.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Enter login info	After receiving the input, the system verifies it.
2	Submit login form	Automatically, the user is logged in and redirected to the dashboard.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	The user inserts the wrong info	An error message is shown by the system.
2	"Forgot Password" is clicked	Verification via email is requested by the system.

This use case outlines the steps for users to log in, detailing both the typical actions taken and the responses for alternate scenarios, as shown in table 4.

Use-Case 3: Manage Budget

Table 5: Manage budget.

Identifier	UC-ManageBudget	
Purpose	To help brides manage their wedding budget.	
Priority	High	
Pre-conditions	The app user has successfully logged in.	
Post-conditions	The user's spending is monitored and controlled.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	The budget section is accessed	The budget interface is shown by the system.
2	Users set budgets	Categories are shown and the budget is saved by the system.
3	User enters expenses	The budget status is updated in real-time by the system.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	The user attempts to exceed the budget	Whenever the user approaches the limit, the system will display an alert.
2	Views visual reports	Graphs and charts are produced by the system.

This use case outlines the steps for managing a wedding budget, detailing the typical actions taken by users and the system's responses to various scenarios, as shown in table 5.

Use-Case 4: Vendor Directory

Table 6: Vendor directory.

Identifier	UC-VendorDirectory	
Purpose	To assist users in finding and managing vendors.	
Priority	High	
Pre-conditions	The app user has successfully logged in.	
Post-conditions	The user can see listed suppliers and get in touch with them.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Accessing vendor directory	A list of suppliers is shown by the system.
2	Category filters are applied	The list is updated by the system according to the filters.
3	Vendor profile is chosen	All the vendor's information is shown in detail by the system.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Searching for a vendor	A matching set of results is shown by the system.

This use case outlines the process for finding and managing vendors, detailing the typical actions users take and the corresponding system responses, as shown in table 6.

Use-Case 5: Task Organization

Table 7: Task organization.

Identifier	UC-TaskOrganization	
Purpose	To help brides coordinate their wedding tasks.	
Priority	High	
Pre-conditions	The app user has successfully logged in.	
Post-conditions	The user receives a timely list of tasks to complete.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Task management is accessed	The list of tasks is shown by the system.
2	New task created	The system adds the task to the list and saves it.
3	The deadline set by the user	A task reminder is set up by the system.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	User completes task	The status of the task is updated by the system.
2	Task Editing	The task can be changed by using the system.

This use case outlines the process for coordinating wedding tasks, detailing the typical actions taken by users and the system's responses, as shown in table 7.

Use-Case 6: Guest List Management

Table 8: Guest list management.

Identifier	UC-GuestlistManagement	
Purpose	To simplify the organization of guests.	
Priority	High	
Pre-conditions	The app user has successfully logged in.	
Post-conditions	The user can manage their guest list.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Accessing the guest list	The interface for the guest list is shown by the system.
2	Guest is added	Guest details are saved by the system.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Edits guest info.	The guest's information is updated by the system.
2	Guests are deleted.	The guest is removed from the list by the system.

This use case outlines the process for organizing and managing a guest list, detailing the typical actions users take and the system's responses, as shown in table 8.

Use-Case 7: Wedding Information

Table 9: Wedding Information

Identifier	UC-WeddingInfo	
Purpose	To allow the user to enter wedding details (date and location) and view a live countdown to the wedding day.	
Priority	Medium	
Pre-conditions	The app user has successfully logged in.	
Post-conditions	The wedding information is saved, and a countdown to the wedding day is displayed.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	The user accesses the "Wedding Info" section	The system displays a form to input wedding date and location.
2	The user enters the wedding date and location	The system saves the entered information.
3	Viewing the countdown	The system calculates and displays the number of days remaining until the wedding date.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	The user edits the wedding date or location	The system updates the information and recalculates the countdown.
2	The user deletes the wedding information	The system removes the details and hides the countdown feature until new information is added.

This use case outlines the process for entering and viewing wedding information, detailing the typical actions users take and the system's responses, as shown in Table 9.

4.3 Nonfunctional Requirements

1. **Performance Requirements**
   - **Response Time:** The app should have a response time of less than 6 seconds for most user interactions, especially critical actions like searching for vendors, making payments, or sending messages.
   - **Scalability:** The app should be able to handle increasing user loads and data volume without significant performance degradation.

2. **Safety Requirements**
   - **Vendor Verification:** Implement a rigorous vendor verification process to protect users from fraudulent or unreliable service providers.
   - **User Education:** Provide clear guidelines and tips to users on how to stay safe and secure while using the app.

3. **Security Requirements**
   - **Data Encryption:** Sensitive user data, such as passwords, financial information, and personal details, should be encrypted using strong encryption algorithms.
   - **Secure Authentication:** Implement robust authentication mechanisms, including strong password requirements.
   - **Data Privacy and Compliance:** Adhere to relevant data privacy regulations and implement measures to protect user data from unauthorized access, breaches, and misuse.
   - **Secure Communication:** Use HTTPS to encrypt communication between the app and the server.

4. **Software Quality Attributes**
   - **Usability:** The app should have a user-friendly interface, intuitive navigation, and clear instructions.
   - **Reliability:** The app should be reliable and free of bugs, ensuring consistent performance and data integrity.
   - **Performance:** The app should have optimal performance, with fast load times and responsive interactions.
   - **Maintainability:** The code should be well-structured, documented, and easily maintainable.

4.4 Analysis Models

- **Use-case diagram:**

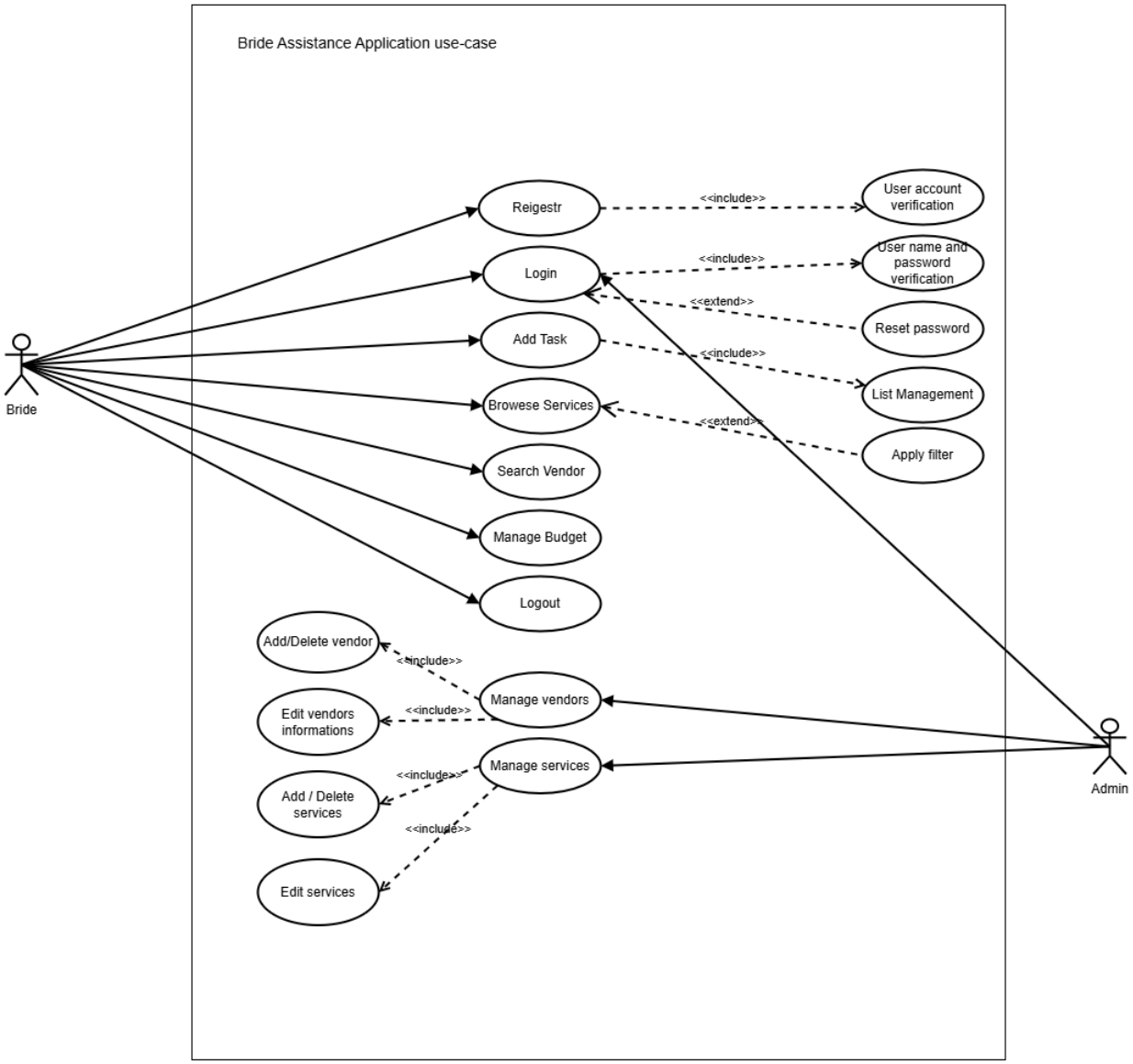

Figure 3: use-case diagram.

This diagram illustrates the various use cases for Bride Assistance Application, showcasing how users interact with different functionalities such as registration, login, task management, vendor management, and budget organization. As shown in figure 3, the diagram provides a clear visual representation of user interactions within the application.

CHAPTER 5: SYSTEM DESIGN

In this chapter, the following design model will be shown. Component diagram, Deployment diagram, Sequence diagram, Complete class diagram, Entity-relationship diagram.

- **Entity-relationship diagram (ERD):**

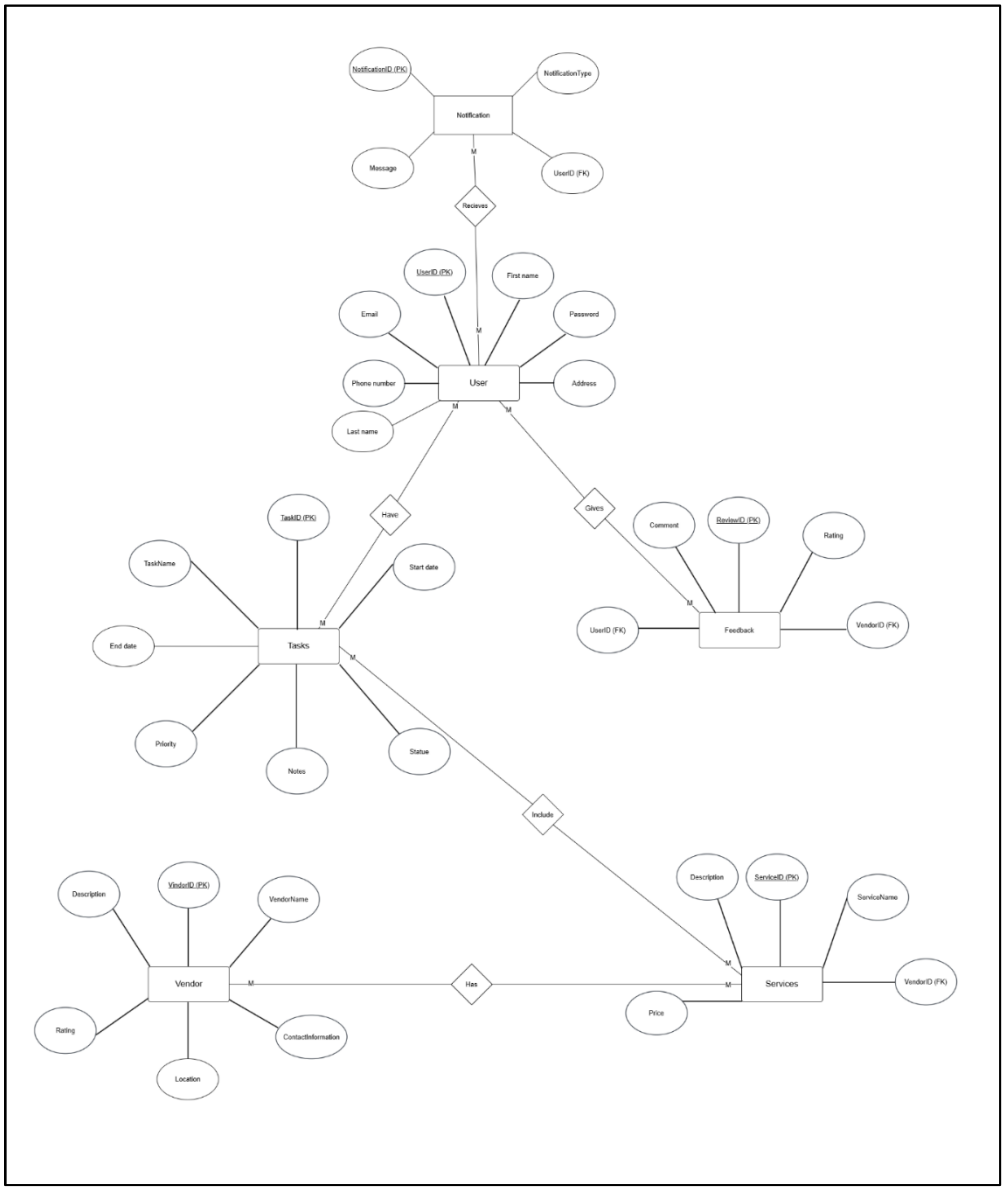

Figure 4: Entity relationship diagram.

The **ERD** for the Bride Assistant App highlights the key components of the system and their connections, as shown in figure 4. **Users** can manage tasks and leave reviews for vendors. **Tasks** help users track their wedding planning activities. **Vendors** provide various services and receive feedback through reviews. **Services** represent the offerings by vendors, while **reviews** capture user

feedback about vendors. These relationships create a simple, user-friendly structure to efficiently organize wedding planning and vendor management.

- **Complete class diagram:**

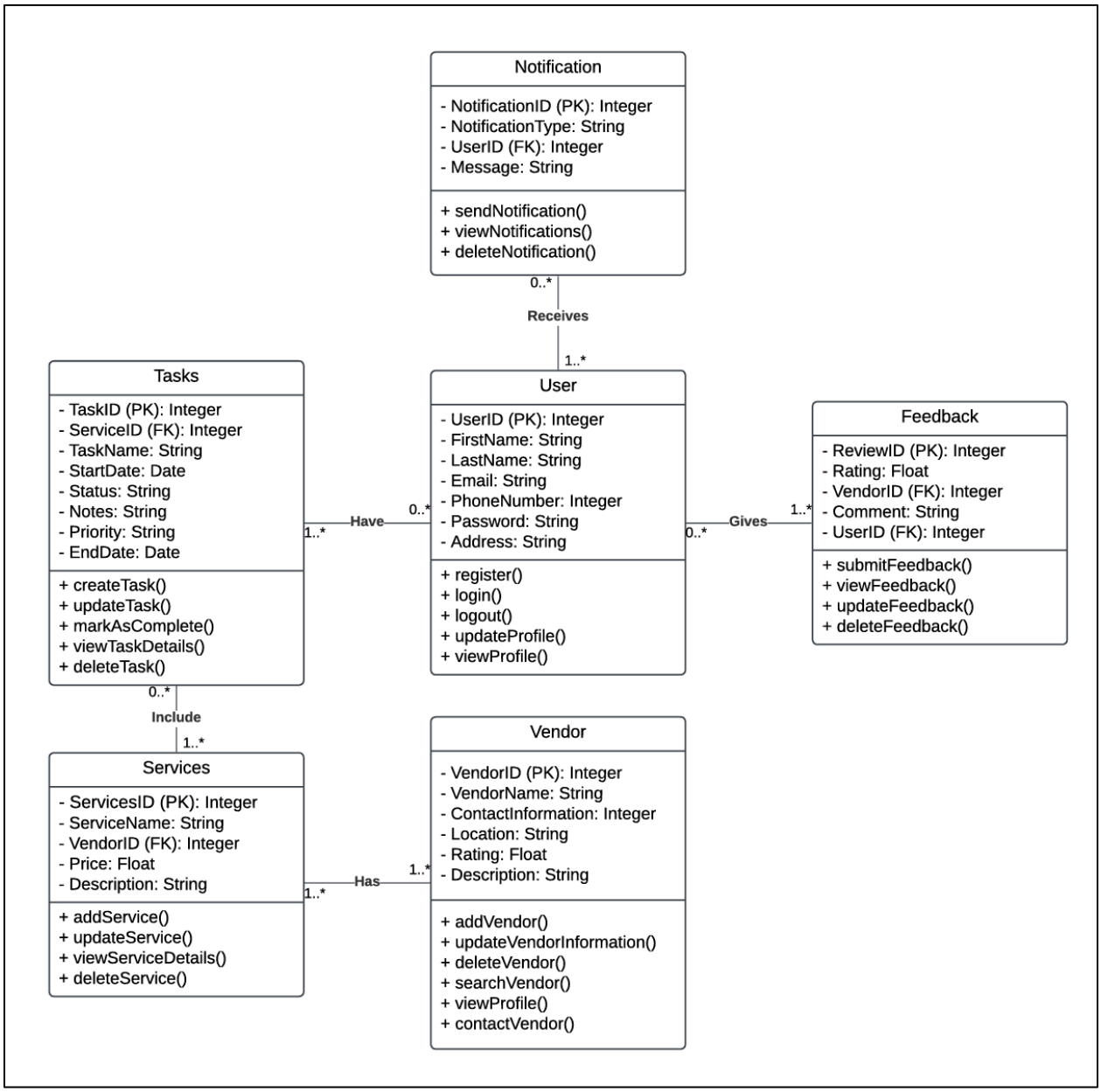

Figure 5: Complete class diagram.

The **class diagram** represents a system that helps manage **users**, **vendors**, **services**, **feedback**, **notifications**, and **tasks**, as shown in figure 5. It includes several key classes: **User**, which stores personal information and has methods for managing accounts; **Services**, which details the services offered and provides ways to add or update them; **Vendor**, which captures vendor details and includes management features; **Feedback**, where users can leave reviews; **Notification**, which handles alerts for users; and **Tasks**, which helps manage tasks related to services. The diagram

also shows how these elements are connected, like users receiving notifications and giving feedback, making it clear how the system works together.

- **Sequence diagram:**

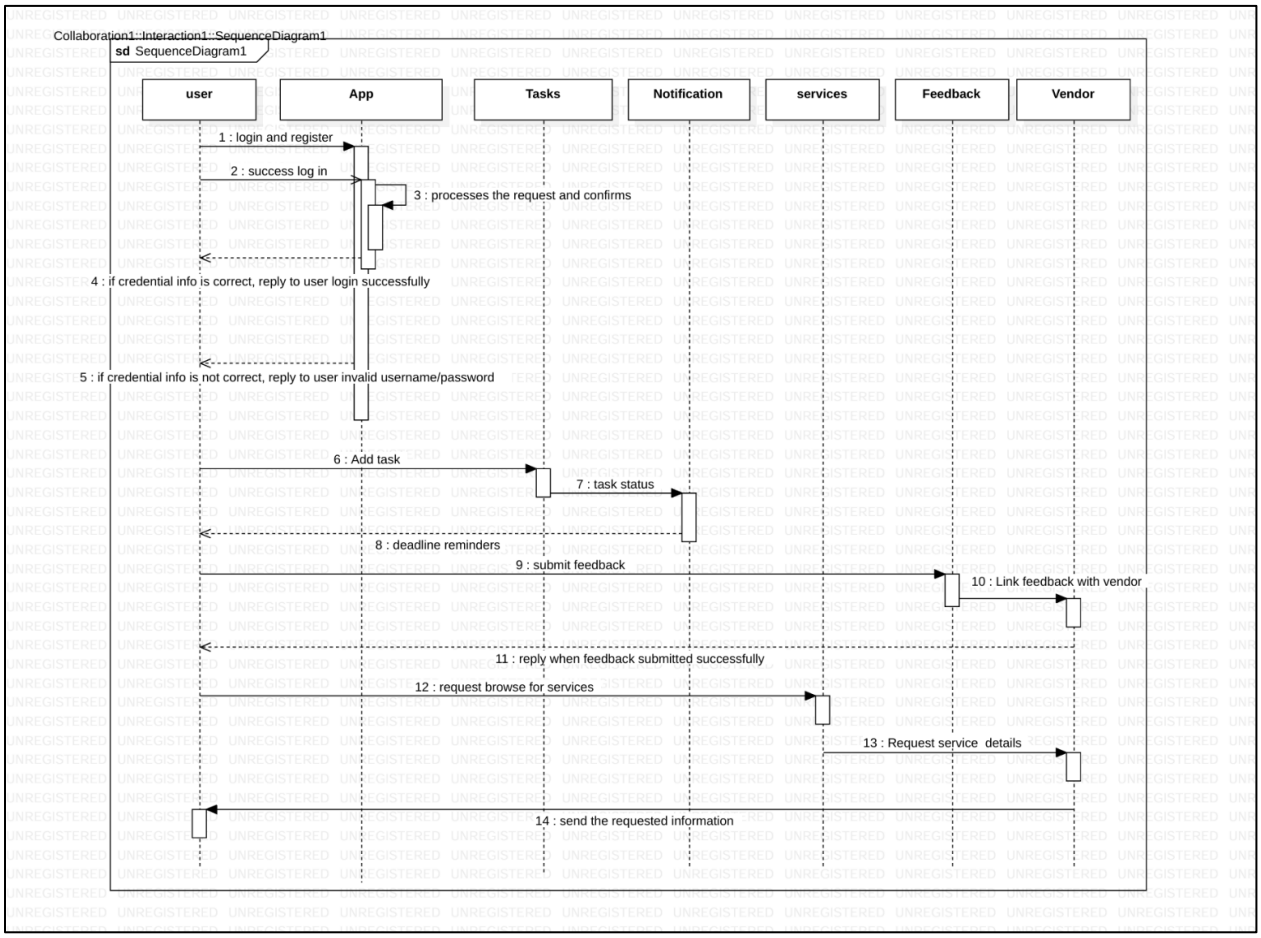

Figure 6: Sequence diagram.

The **sequence diagram** shows how a system's login, task management, feedback submission, and service browsing functions interact, as shown in figure 6. When users login or register, the application checks their credentials and either grants access or produces an error if their information is incorrect. The notification feature allows users to add tasks, keep track of their statuses, and get reminders when deadlines are approaching. The user is notified when their feedback has been successfully sent and linked to the appropriate vendor. Users can request specific information for service browsing, and the program will retrieve and provide it. The application acts as the focal point of the process, coordinating with other elements such as tasks, notifications, feedback, services, and vendors. Although the system efficiently manages simple interactions and guarantees seamless functioning, additional considerations for error handling and parallel processing could enhance its robustness.

- **Component diagram:**

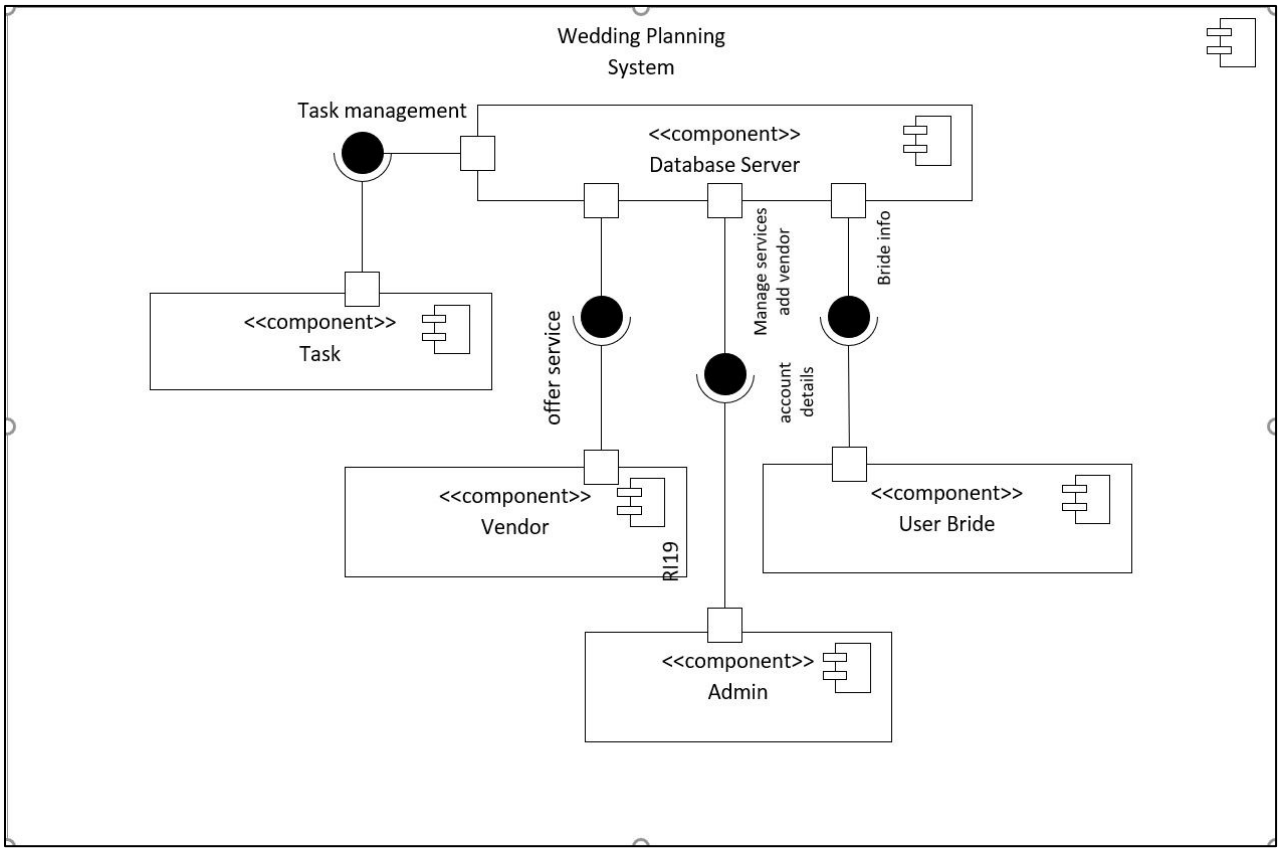

Figure 7: Component diagram.

This **component diagram** represents a Wedding Planning System, showcasing its main modules and interactions, as shown in F 7. The central Database Server component connects with other components, including Task, Vendor, User Bride, and Admin, to handle core functionalities. The admin component manages services and vendors, while the Vendor component offers services. The User Bride component interacts with the system to provide bride-related information. Task management is a dedicated module for organizing and managing wedding-related tasks efficiently.

- **Deployment diagram:**

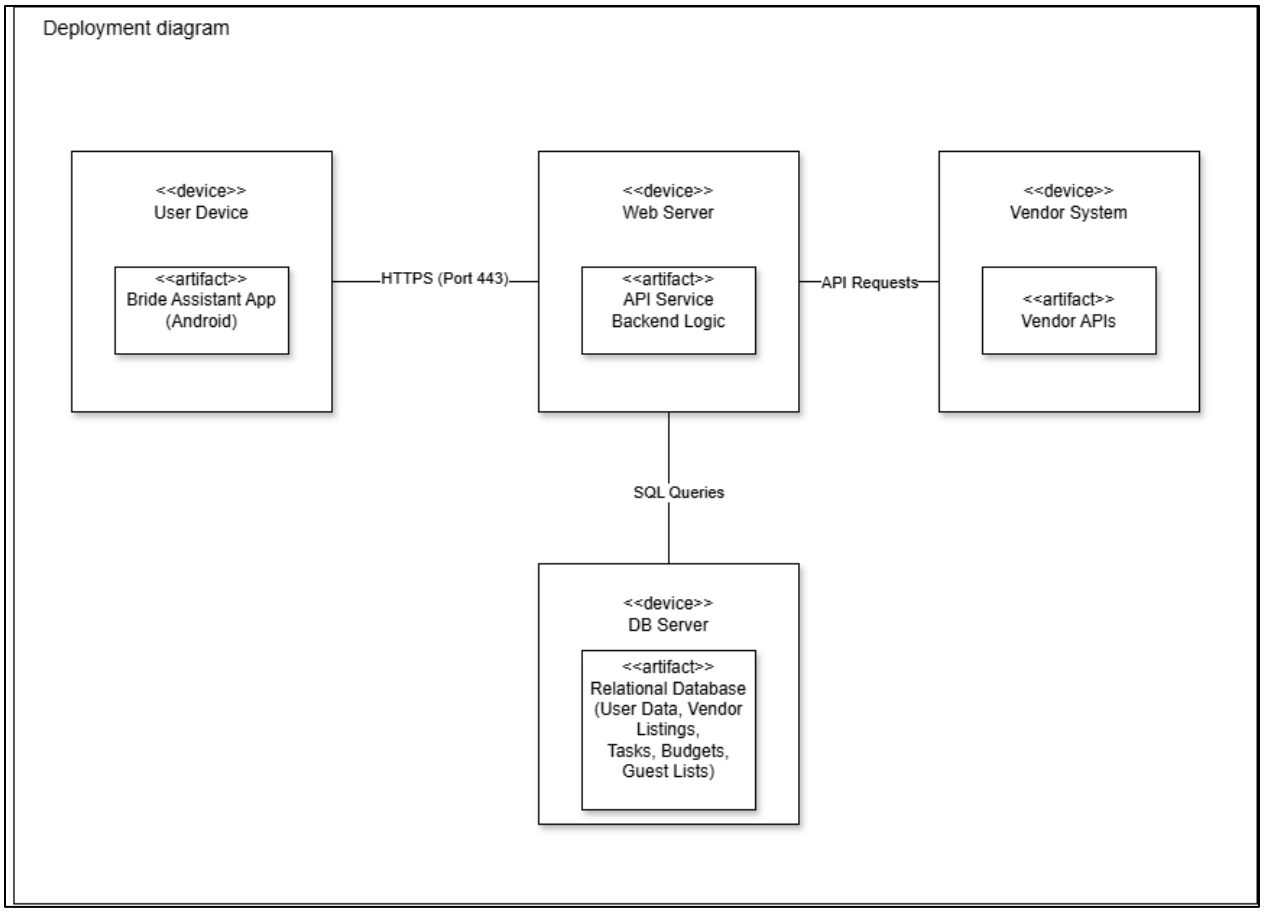

Figure 8: Deployment diagram.

The **deployment diagram** illustrates how the Bride Assistant App operates, as shown in figure 8. The **User Device** runs the app on a smartphone (Android) and securely connects to the web server via HTTPS (Port 443). The **Web Server** processes requests from the app, manages business logic, and communicates with both the database and external vendor systems. The **Database Server** stores key information, such as user data, vendor listings, tasks, budgets, and guest lists, accessed by the web server through SQL queries. The **Vendor System** represents external platforms providing services, including vendor details and bookings, connected via APIs. This architecture ensures secure communication between users, the backend system, and external vendors, facilitating smooth and efficient wedding planning.

CHAPTER 6: SYSTEM IMPLEMENTATION

6.1 Software requirements:

- Android Studio for programming
- Firebase for the database
- BlueStack App Player to emulate the application

6.2 Hardware requirements:

- Processor Intel(R) Core(TM) i5-1035G4 CPU @ 1.10GHz 1.50 GHz
- RAM 8.00 GB
- System type 64-bit operating system, x64-based processor
- Edition Windows 11 Home
- Version 24H2
- OS build 26100.3037

6.3 Programming language:

- Java

6.4 Software Interfaces:

- **Admin Interfaces:**

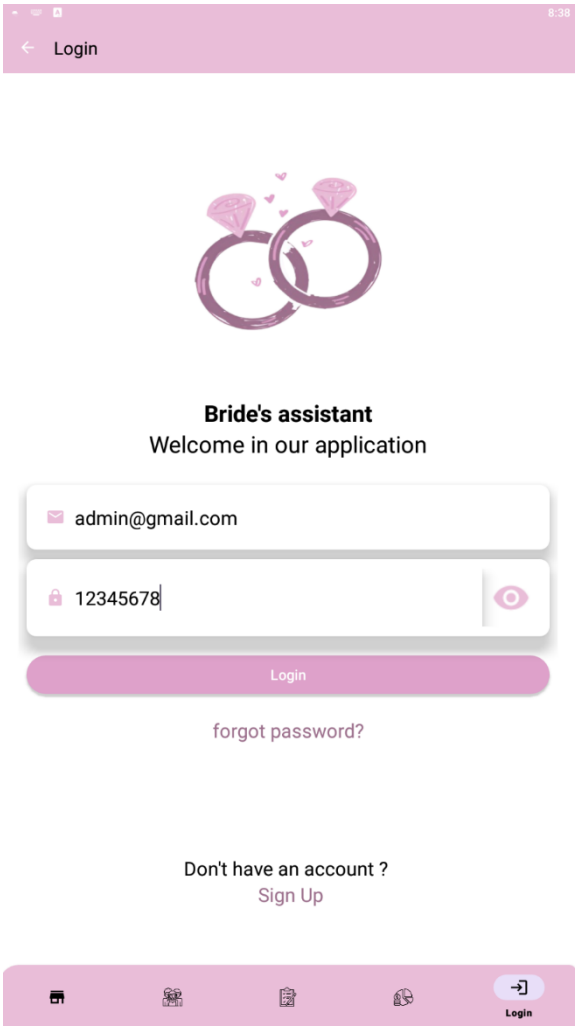

Figure 9: Admin login screen.

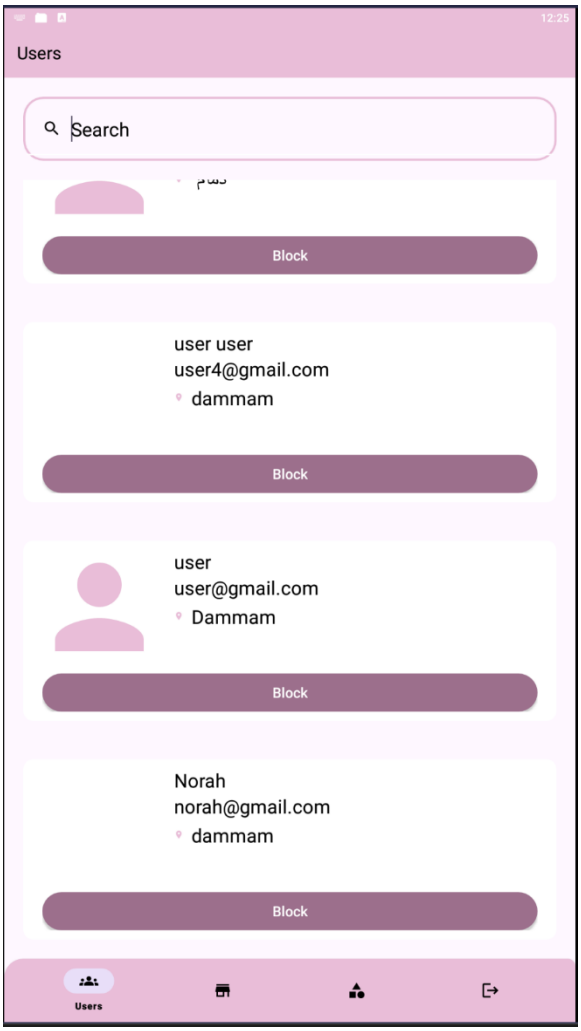

Figure 10: Users list.

The administrative section of the Bride Assistant application features a login interface for secure access and a "Users list" for the management of registered users. The login screen necessitates administrator credentials, while the "Users list" offers a searchable directory of user accounts, enabling administrative actions such as blocking access. This structure facilitates effective user oversight within the application.

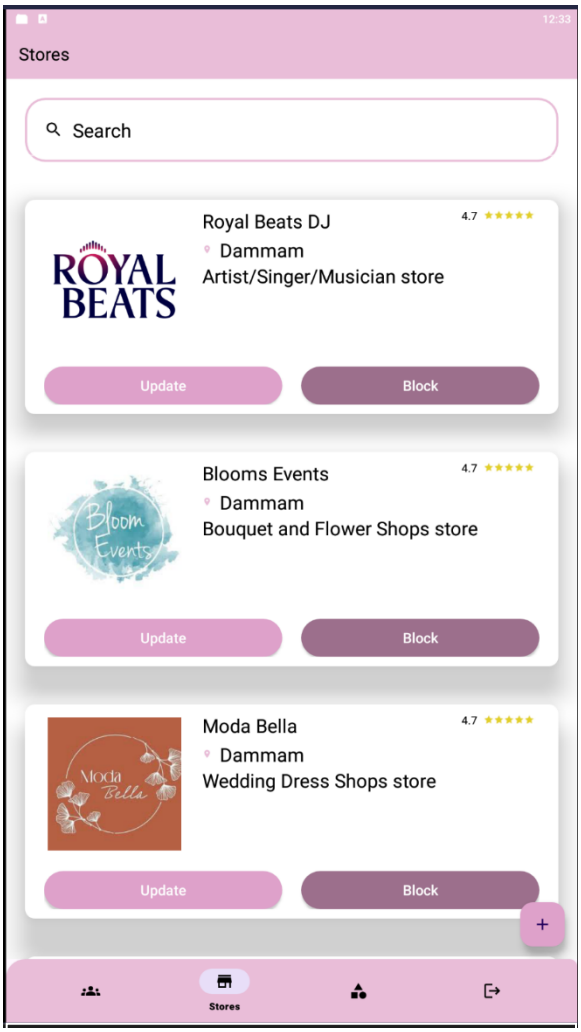

Figure 11: "Stores" list.

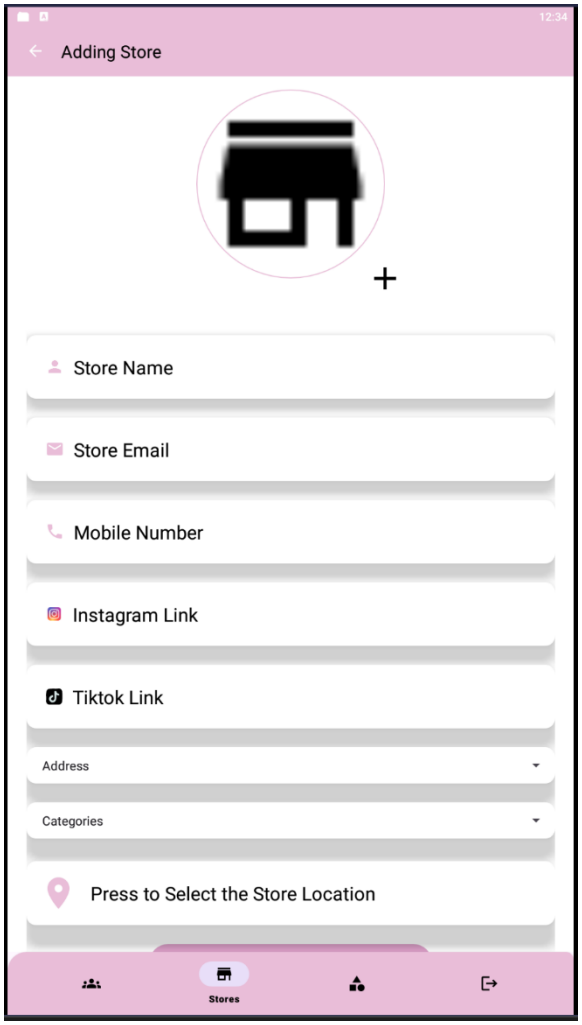

Figure 12: "Adding Store" screen.

The **"Stores"** interface serves as the central management hub for registered vendors and service providers within Bride Assistant App. It presents a browsable and searchable list of existing stores, each with key information and immediate actions for updating or temporarily blocking their visibility, as shown in Figure 11. The Floating Action Button (FAB)**,** prominently displayed, provides administrators with direct access to the **"Adding Store"** interface, as shown in Figure 12. The "Adding Store" interface offers a dedicated form for onboarding new vendors by inputting essential details such as contact information, social media links, location, and business category, thereby facilitating the expansion of the app's directory of wedding-related services within the specified region.

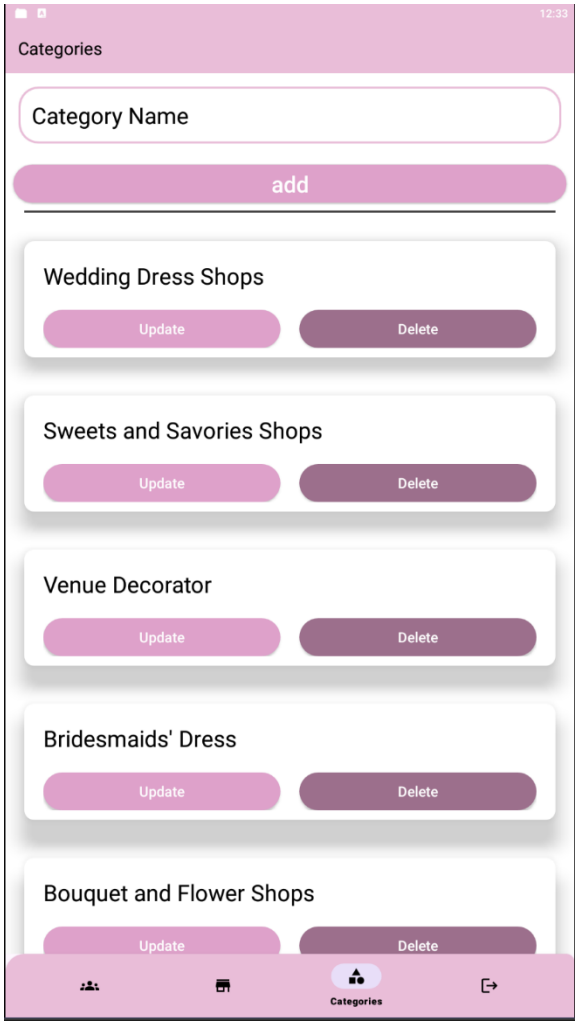

Figure 13: " Categories" screen.

The **"Categories"** management screen within the Admin Panel is essential for organizing vendors and services within Bride Assistant App. It provides administrators with the necessary tools to define, manage, and maintain a clear and comprehensive categorization system, as shown in Figure 13. This enables efficient browsing and discovery of wedding-related services by users and ensures the platform's adaptability to evolving needs and a growing vendor base through the ability to add, update, and delete categories.

- **User Interfaces:**

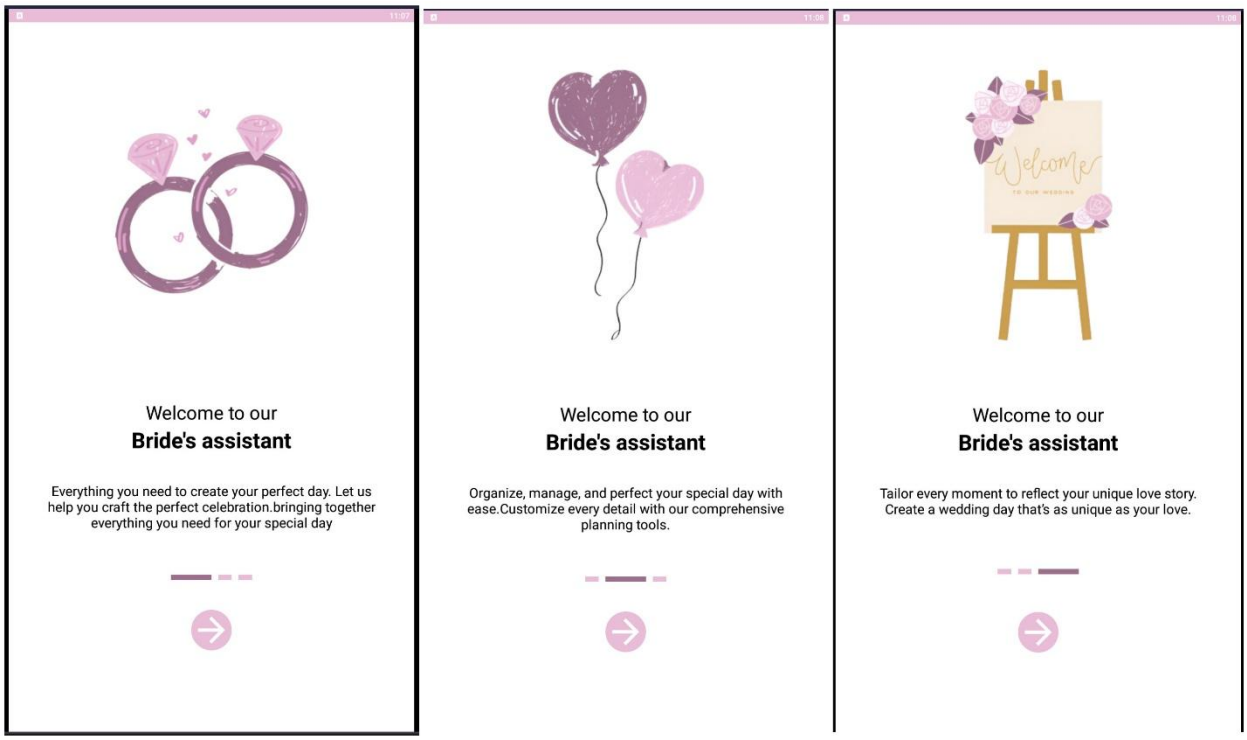

Figure 14 : Welcoming screens.

The initial user interface presents a series of "**welcoming screens"** designed to introduce the Bride Assistant App and its fundamental value proposition to prospective users, primarily brides. These introductory screens serve as the app's first impression, artfully communicating its core functionalities and the inherent benefits it offers for navigating the wedding planning journey, as shown in Figure 14. Through a visually appealing aesthetic and thoughtfully crafted messaging, these screens establish a positive and encouraging tone, setting the stage for a seamless and personalized experience in creating their special day.

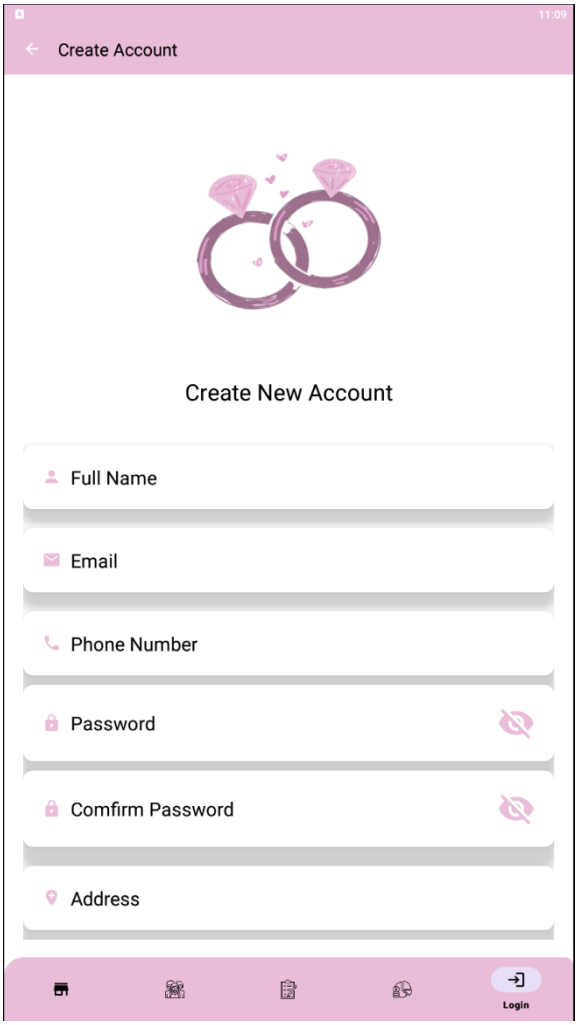

Figure 15 : Create account screen.

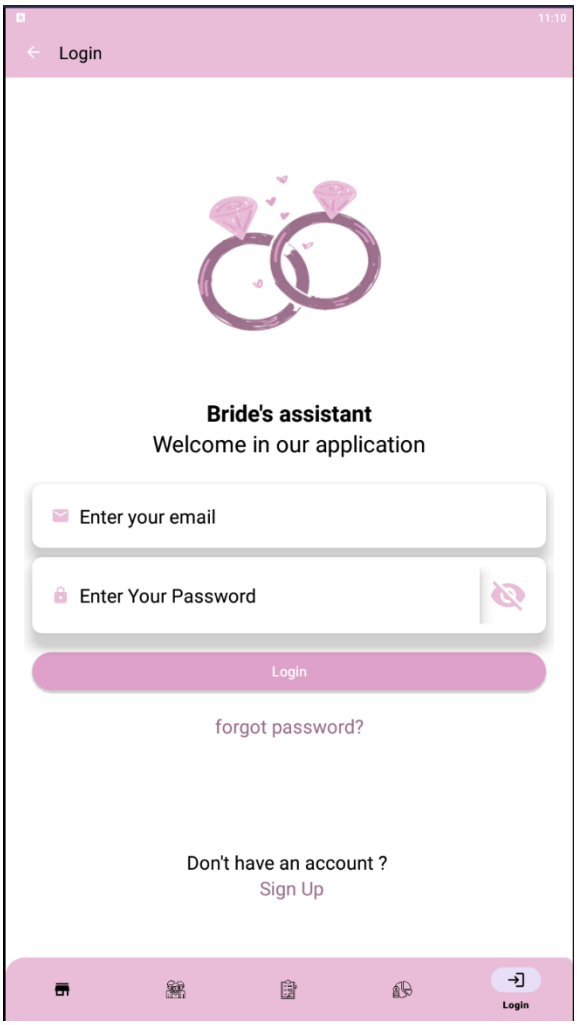

Figure 16 : Login screen.

The initial access to the Bride Assistant application is facilitated through two key user interfaces: **"Create Account"** and **"Login."** The **"Create Account"** screen serves as the registration point for new users, presenting a clear form for essential personal details to establish their profile and commence their personalized wedding planning journey, as shown in Figure 15. Conversely, the **"Login"** screen provides returning users with a direct and simplified pathway to access their existing profiles and previously saved wedding plans, prioritizing efficient entry to continue their planning process, as shown in Figure 16. Both interfaces incorporate visual cues and security features to ensure a user-friendly and secure initial interaction with the application.

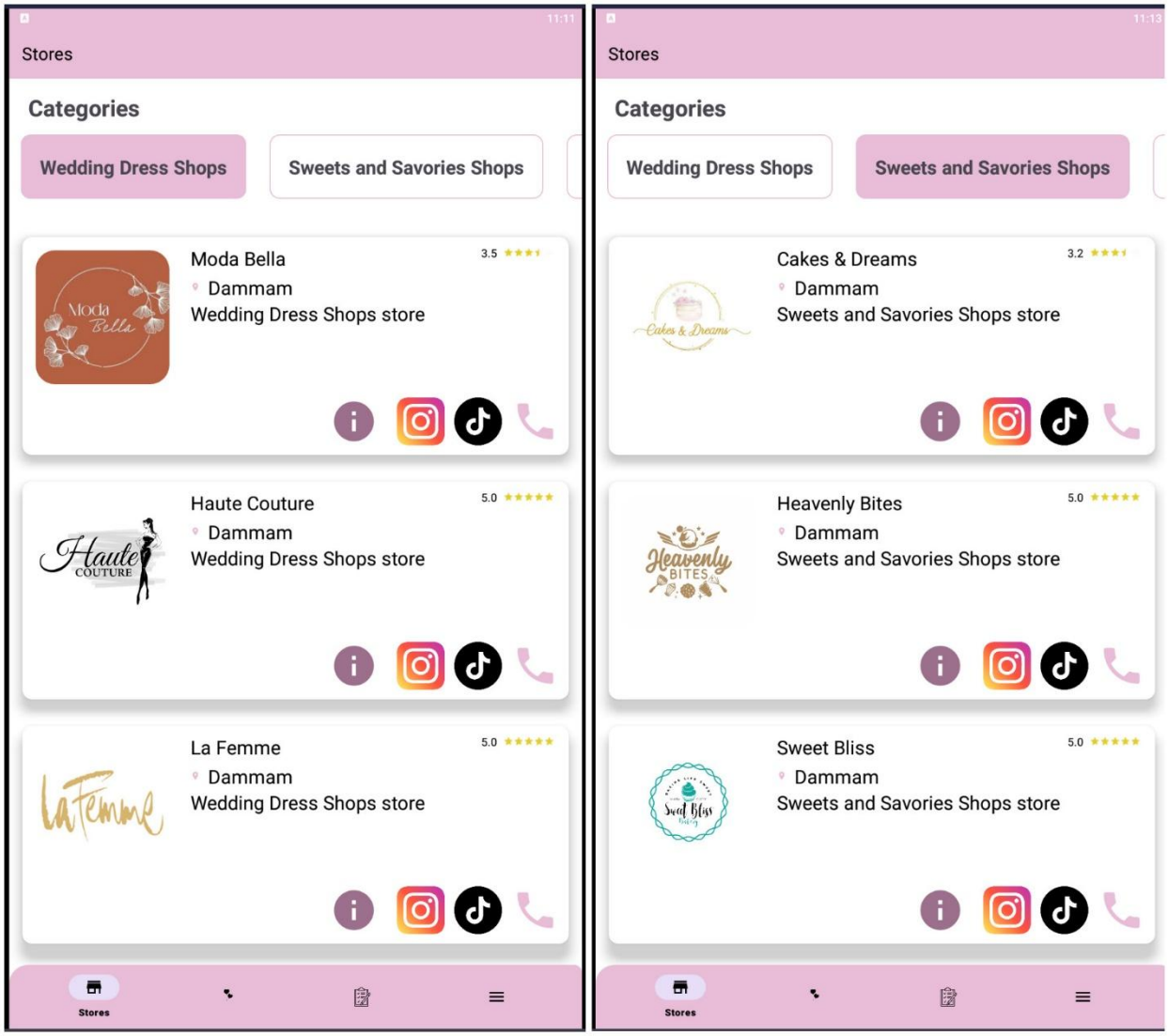

Figure 17 : Stores screen.

This **"Stores"** screen, organized by categories, provides a user-friendly way for brides to discover and connect with various wedding vendors. The direct-action icons streamline the process of getting more information, viewing social media, and contacting the stores, as shown in Figure 17.

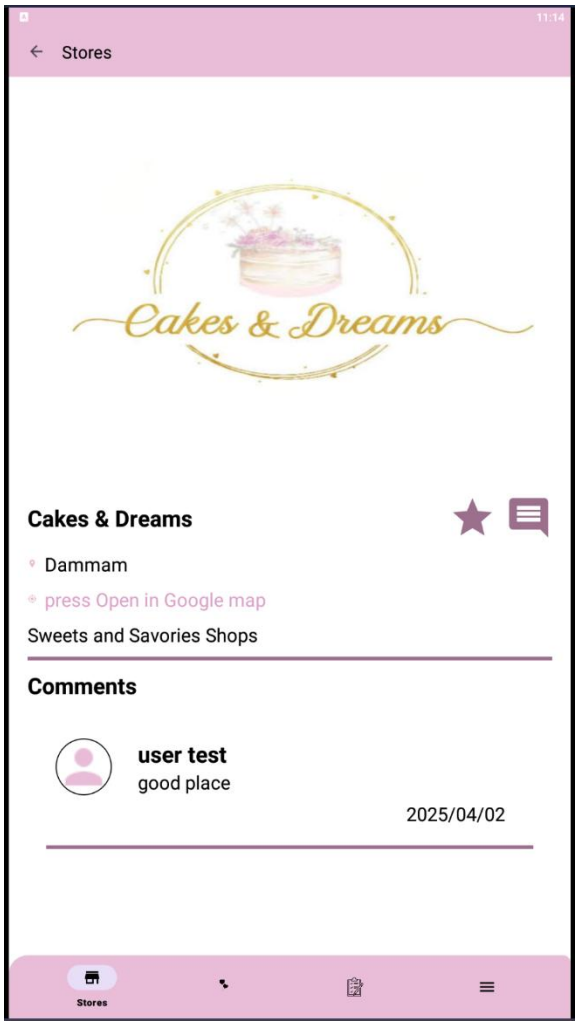

Figure 18 : Detailed profile screen.

The detailed profile page for a specific vendor, exemplified by "Cakes & Dreams," serves as a vital connection point within the Bride Assistant App. Accessed for in-depth information, this page provides essential details, location via Google Maps integration, and social proof through ratings and user comments, as shown in Figure 18. Empowering brides to make informed decisions and directly engage with potential vendors.

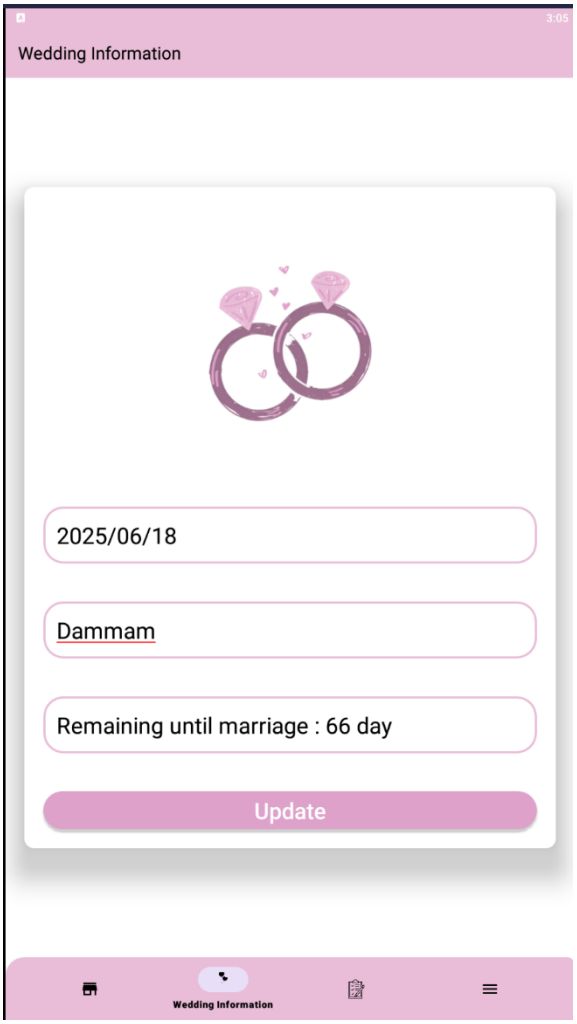

Figure 19 : Wedding information screen.

This **"Wedding Information"** screen allows the bride to manage the wedding date and location. It displays the current selection and includes a countdown to the wedding day, as shown in Figure 19. An "Update" button saves any changes made.

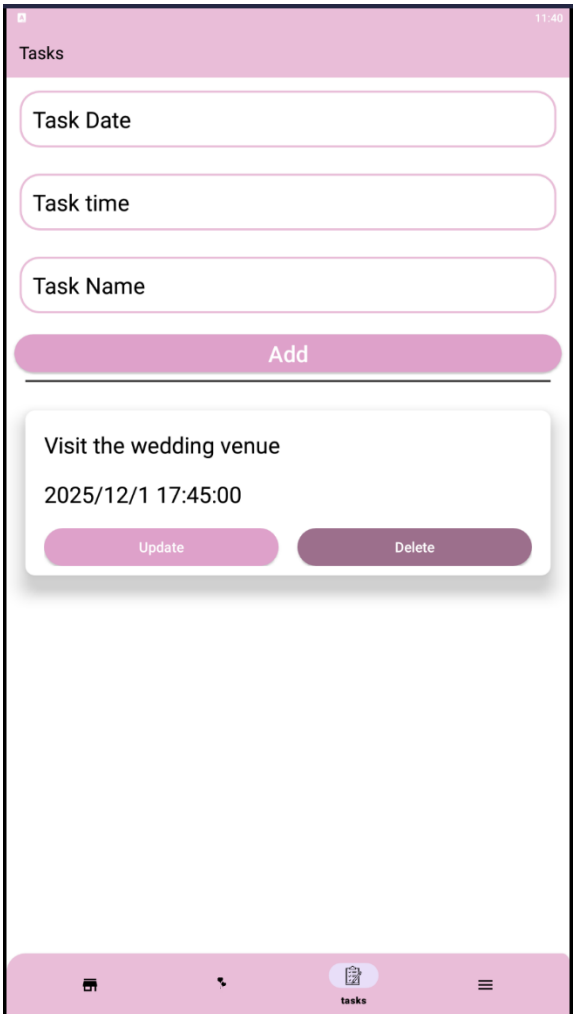

Figure 20 : Tasks screen.

The **"Tasks"** screen is a vital organizational tool within the Bride Assistant App, empowering the bride to stay on top of her wedding planning timeline and ensure all necessary steps are completed. The ability to add, update, and delete tasks provides flexibility and control over the planning process, as shown in Figure 20.

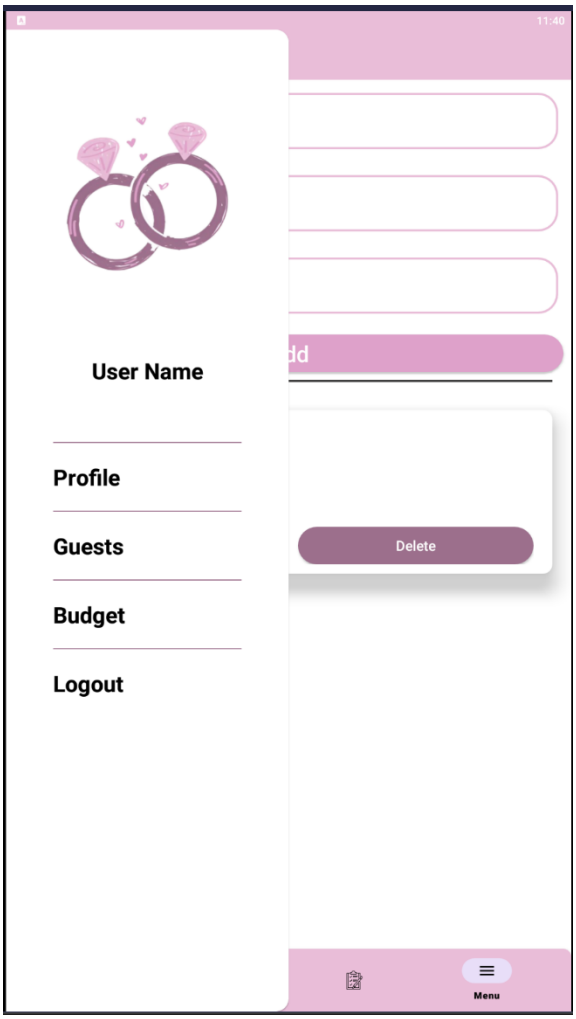

Figure 21 : Menu screen.

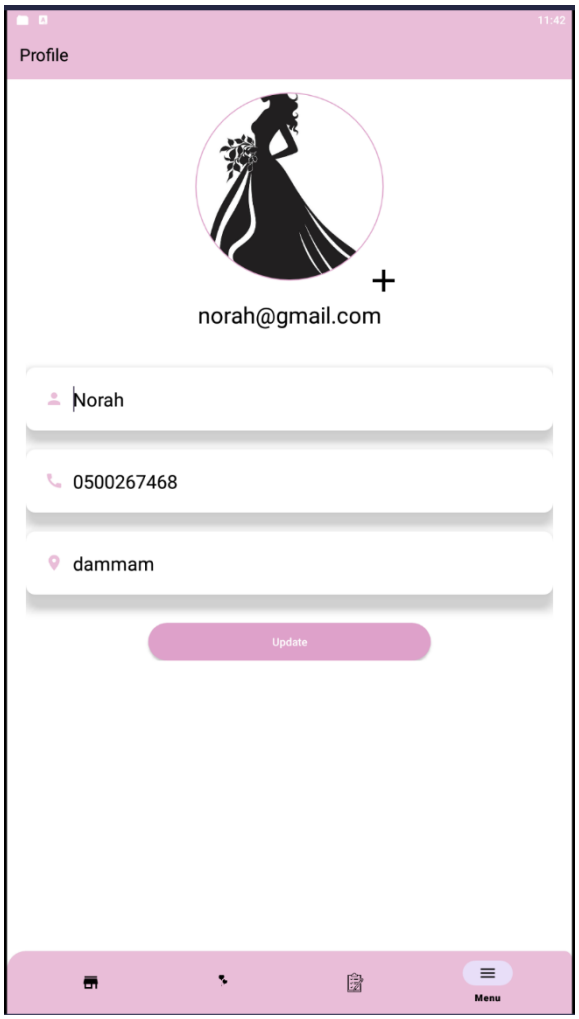

Figure 22 : User profile screen.

The **Menu** section serves as the primary navigation hub within the Bride Assistant application, providing users with access to its core functionalities and features, as shown in Figure 21. Through a clearly organized structure, it enables seamless transitions between various planning tools, information resources, and community engagement areas, ensuring a streamlined and intuitive user experience.

While the **User Profile** section centralizes the bride's personalized experience within the application. It allows for the management of personal details, as shown in Figure 22. Including contact information and profile customization. Furthermore, it provides access to key planning tools such as guest list management and budget tracking, enabling the bride to efficiently organize and oversee critical aspects of her wedding preparations.

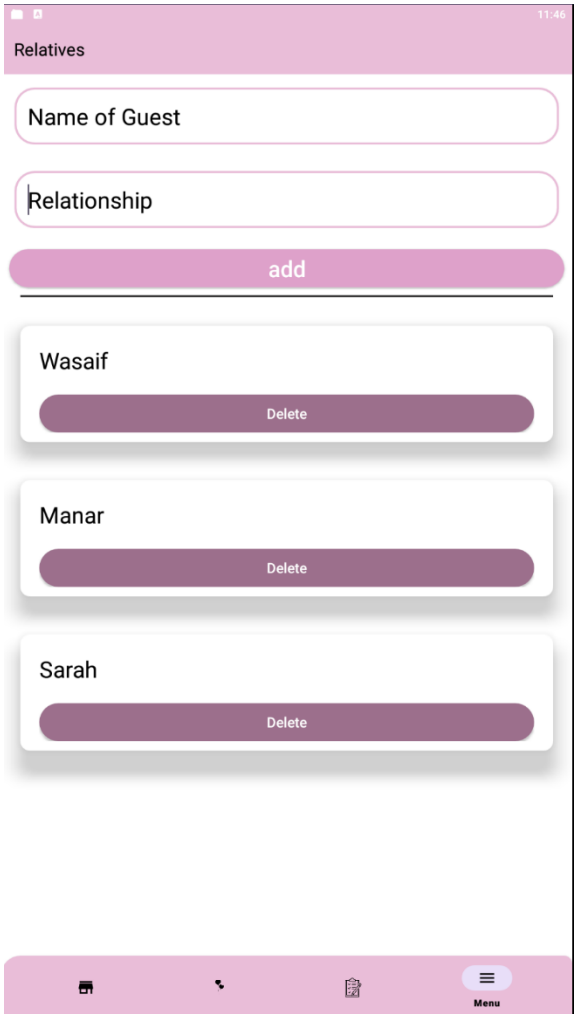

Figure 23 : Guests screen.

This interface provides a dedicated platform for the user to manage their wedding guests. It allows for the systematic addition of relatives by name and relationship, as well as the subsequent removal of individuals from the attendance list as required, as shown in Figure 23.

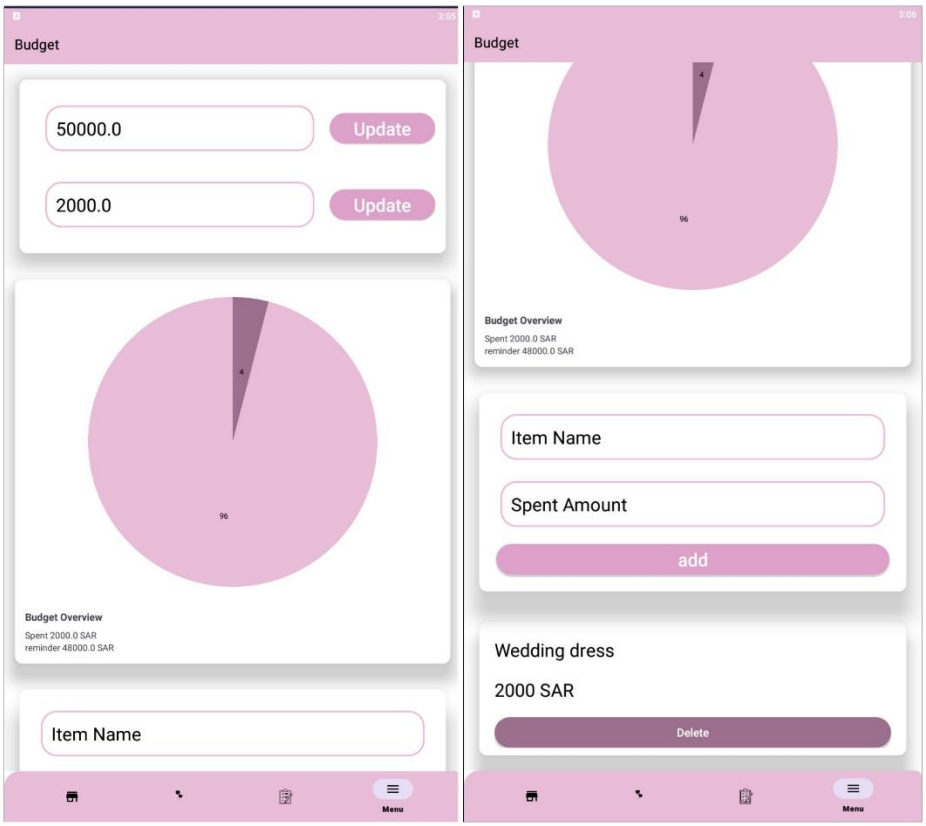

Figure 24 : Budget screen.

The "**Budget"** screen allows users to establish and monitor their wedding finances. It facilitates the allocation of funds to different expense categories and the detailed tracking of individual expenditures, as shown in Figure 24. A visual representation of the budget distribution is provided through a pie chart, complemented by real-time updates on spent and remaining amounts. This feature enables users to maintain a clear understanding of their financial status throughout the wedding planning process with the capacity to add and remove specific expenses for accurate management.

- **Code snippets**

1- <u>Sign up code:</u>

```
package com.app.bridesassistant.UI.Authentication;

import android.content.Intent;
import android.os.Bundle;
import android.text.InputType;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ArrayAdapter;
import android.widget.CompoundButton;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.navigation.Navigation;

import com.app.bridesassistant.DataClass.User;
import com.app.bridesassistant.R;
import com.app.bridesassistant.Repo.UsersRepo;
import com.app.bridesassistant.UI.User.UserMainActivity;
import com.app.bridesassistant.Utils.Content;
import com.app.bridesassistant.Utils.Dialoge.SweetDialog;
import com.app.bridesassistant.Utils.Validation.EmailValidator;
import com.app.bridesassistant.Utils.Validation.PasswordValidation;
import com.app.bridesassistant.Utils.Validation.ValidationPhoneNumber;
import com.app.bridesassistant.databinding.FragmentRegistrationBinding;
import com.bumptech.glide.Glide;
import com.google.firebase.auth.FirebaseAuth;

import java.util.ArrayList;

import cn.pedant.SweetAlert.SweetAlertDialog;

public class RegistrationFragment extends Fragment {
    private FragmentRegistrationBinding mBinding;
    private int pStatus = 1, pStatus2 = 1;
    private SweetAlertDialog loading;
    FirebaseAuth auth;

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        mBinding = FragmentRegistrationBinding.inflate(inflater,
container, false);

        auth = FirebaseAuth.getInstance();
        mBinding.createAccount.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
```

```
                    checkEnteredUserData();

            }
        });
        showPassword();
        showPassword2();
        return mBinding.getRoot();
    }

    private void checkEnteredUserData() {
        String name = mBinding.useName.getText().toString();
        String email = mBinding.userEmail.getText().toString();
        String password = mBinding.userPassword.getText().toString();
        String confirmPassword =
mBinding.userConfirem.getText().toString();
        String phone = mBinding.userPhoneNumber.getText().toString();
        String address = mBinding.userAddress.getText().toString();
        if (name.isEmpty()) {
            mBinding.useName.setError("Add username");
        } else if (!EmailValidator.isValidEmail(email)) {
            mBinding.userEmail.setError("Please enter correct email
'user@gmail.com'");
        } else if (!ValidationPhoneNumber.validatePhoneNumber(phone)) {
            mBinding.userPhoneNumber.setError("Please make sure the number
is correct as it starts with (05) and consists of 10 digits.");
        } else if (!PasswordValidation.validatePassword(password)) {
            mBinding.userPassword.setError("Please enter a strong password
that contains uppercase letters, lowercase letters, numbers and symbols
(at least 8 characters)");
        } else if (!confirmPassword.equals(password)) {
            mBinding.userConfirem.setError("Password does not match");
        } else if (address.isEmpty()) {
            mBinding.userAddress.setError("Add the title");
        } else {
            funLoading();
            User user = new User(name, email, password, phone, address,
"", "", false);
            createAccount(email, password, user);
        }
    }

    private void createAccount(String email, String password, User user) {
        auth.createUserWithEmailAndPassword(email, password)
                .addOnSuccessListener(authResult -> {

user.setUserId(authResult.getUser().getUid().toString());
                    saveUserData(user);
                })
                .addOnFailureListener(e -> {
                    loading.dismiss();
                    funFailed("Failed to create account Please change your
email");
                });
    }

    private void saveUserData(User user) {
```

```
        UsersRepo repo = new UsersRepo();
        repo.saveUserData(user).observe(getViewLifecycleOwner(), status ->
{
            loading.dismiss();
            if (status.equals("failed")) {
                funFailed("Failed to create account Please change your
email");
            } else {
                funSuccessfully("New account created successfully");
            }
        });
    }

    private void showPassword() {
        mBinding.showPassword.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                if (pStatus == 1) {
                    Glide.with(getActivity())
                            .load(R.drawable.baseline_visibility_24)
                            .centerCrop()
                            .into(mBinding.showPassword);
                    pStatus = 0;

mBinding.userPassword.setInputType(InputType.TYPE_CLASS_TEXT);
                } else {
                    Glide.with(getActivity())
                            .load(R.drawable.baseline_visibility_off_24)
                            .centerCrop()
                            .into(mBinding.showPassword);
                    pStatus = 1;

mBinding.userPassword.setInputType(InputType.TYPE_CLASS_TEXT |
InputType.TYPE_TEXT_VARIATION_PASSWORD);
                }
            }
        });
    }

    private void showPassword2() {
        mBinding.showPassword2.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                if (pStatus2 == 1) {
                    Glide.with(getActivity())
                            .load(R.drawable.baseline_visibility_24)
                            .centerCrop()
                            .into(mBinding.showPassword2);
                    pStatus2 = 0;

mBinding.userConfirem.setInputType(InputType.TYPE_CLASS_TEXT);
                } else {
                    Glide.with(getActivity())
                            .load(R.drawable.baseline_visibility_off_24)
```

```
                            .centerCrop()
                            .into(mBinding.showPassword2);
                    pStatus2 = 1;

mBinding.userConfirem.setInputType(InputType.TYPE_CLASS_TEXT |
InputType.TYPE_TEXT_VARIATION_PASSWORD);
                }
            }
        });
    }





    private void funLoading() {
        loading = SweetDialog.loading(getContext());
        loading.show();
    }

    private void funSuccessfully(String title) {
        SweetAlertDialog success = SweetDialog.success(getContext(),
title);
        success.show();
        success.setConfirmClickListener(new
SweetAlertDialog.OnSweetClickListener() {
            @Override
            public void onClick(SweetAlertDialog sweetAlertDialog) {
                success.dismiss();

Navigation.findNavController(getActivity(),R.id.nav_host_fragment_activity
_auth).navigate(R.id.action_registration_to_login);

            }
        });
    }

    private void funFailed(String message) {
        SweetAlertDialog failed = SweetDialog.failed(getContext(),
message);
        failed.show();
        failed.setConfirmClickListener(new
SweetAlertDialog.OnSweetClickListener() {
            @Override
            public void onClick(SweetAlertDialog sweetAlertDialog) {
                failed.dismiss();
            }
        });
    }


    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle
savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);
        mBinding.signUpLink.setOnClickListener(new View.OnClickListener()
```

```
{
            @Override
            public void onClick(View v) {

Navigation.findNavController(v).navigate(R.id.action_registration_to_login
);

            }
        });
    }
}
```

2- log in code:

```
package com.app.bridesassistant.UI.Authentication;

import android.content.Intent;
import android.os.Bundle;
import android.os.Handler;
import android.text.InputType;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
import androidx.navigation.Navigation;

import com.app.bridesassistant.R;
import com.app.bridesassistant.UI.Admin.AdminMainActivity;
import com.app.bridesassistant.UI.User.UserMainActivity;
import com.app.bridesassistant.Utils.Content;
import com.app.bridesassistant.Utils.Dialoge.SweetDialog;
import com.app.bridesassistant.databinding.FragmentLoginBinding;
import com.bumptech.glide.Glide;
import com.google.android.gms.tasks.OnFailureListener;
import com.google.android.gms.tasks.OnSuccessListener;
import com.google.firebase.auth.AuthResult;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.ValueEventListener;

import cn.pedant.SweetAlert.SweetAlertDialog;

public class LoginFragment extends Fragment {
    private FragmentLoginBinding mBinding;
    private FirebaseAuth auth;
    private int pStatus=1;
    private SweetAlertDialog loading;
    private boolean isLogin=false;
    private DatabaseReference database;

    @Override
```

```
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        mBinding = FragmentLoginBinding.inflate(inflater, container,
false);

        auth = FirebaseAuth.getInstance();
        mBinding.signinBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                if
(mBinding.signInEmail.getText().toString().equals("admin@gmail.com")&&mBin
ding.signInPassword.getText().toString().equals("12345678"))
                    startActivity(new Intent(getActivity(),
AdminMainActivity.class));
                else{
                    checkEnteredData();
                }
            }
        });
        showPassword();
        return mBinding.getRoot();
    }
    private void checkEnteredData()
    {
        String email=mBinding.signInEmail.getText().toString();
        String password=mBinding.signInPassword.getText().toString();
        if (email.isEmpty())
        {
            mBinding.signInEmail.setError("Enter your email");
        }
        else if (password.isEmpty())
        {
            mBinding.signInPassword.setError("Enter your password");
        }
        else{
            funLoading();
            checkIfUser(email,password);
        }
    }
    private void checkIfUser(String email,String password)
    {

auth.signInWithEmailAndPassword(email,password).addOnSuccessListener(new
OnSuccessListener<AuthResult>() {
            @Override
            public void onSuccess(AuthResult authResult) {

checkIfUserBlock(authResult.getUser().getUid().toString());
            }
        }).addOnFailureListener(new OnFailureListener() {
            @Override
            public void onFailure(@NonNull Exception e) {
                loading.dismiss();
                funFailed("Sorry, we couldn't find your account.");
```

```
            }
        });
    }
    private void checkIfUserBlock(String id) {
        database=FirebaseDatabase.getInstance().getReference();

database.child(Content.userTable).child(id).addValueEventListener(new
ValueEventListener() {
            @Override
            public void onDataChange(@NonNull DataSnapshot snapshot) {
                if (snapshot.exists()&&!isLogin) {
                    loading.dismiss();
                    boolean status =
Boolean.parseBoolean(snapshot.child("status").getValue().toString());
                    if (status) {
                        funFailed("Sorry, the administrator has banned
your account from the application.");
                    } else {
                        isLogin=true;
                        Content.type=1;
                        Content.categoryNum=0;
                        funSuccessfully("You have successfully logged
in.");
                        startActivity(new Intent(getActivity(),
UserMainActivity.class));
                    }
                }
            }

            @Override
            public void onCancelled(@NonNull DatabaseError error) {

            }
        });
    }

    private void showPassword() {
        mBinding.showPassword.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {
                if (pStatus == 1) {
                    Glide.with(getActivity())
                            .load(R.drawable.baseline_visibility_24)
                            .centerCrop()
                            .into(mBinding.showPassword);
                    pStatus = 0;

mBinding.signInPassword.setInputType(InputType.TYPE_CLASS_TEXT);
                } else {
                    Glide.with(getActivity())
                            .load(R.drawable.baseline_visibility_off_24)
                            .centerCrop()
                            .into(mBinding.showPassword);
                    pStatus = 1;
```

```
mBinding.signInPassword.setInputType(InputType.TYPE_CLASS_TEXT |
InputType.TYPE_TEXT_VARIATION_PASSWORD);
                }
            }
        });
    }
    private void funLoading() {
        loading = SweetDialog.loading(getContext());
        loading.show();
    }
    private void funSuccessfully(String title) {
        SweetAlertDialog success = SweetDialog.success(getContext(),
title);
        success.show();
    }

    private void funFailed(String message) {
        SweetAlertDialog failed = SweetDialog.failed(getContext(),
message);
        failed.show();
        failed.setConfirmClickListener(new
SweetAlertDialog.OnSweetClickListener() {
            @Override
            public void onClick(SweetAlertDialog sweetAlertDialog) {
                failed.dismiss();
            }
        });
    }

    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle
savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);
        mBinding.signUpLink.setOnClickListener(new View.OnClickListener()
{
            @Override
            public void onClick(View v) {

Navigation.findNavController(v).navigate(R.id.action_login_to_registration
);
            }
        });
        mBinding.forgetPassword.setOnClickListener(new
View.OnClickListener() {
            @Override
            public void onClick(View v) {

Navigation.findNavController(v).navigate(R.id.action_login_to_resetPasswor
d);
            }
        });
    }
}
```

3- View stores code:

```
package com.app.bridesassistant.UI.CommonUi;

import android.Manifest;
import android.app.AlertDialog;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.net.Uri;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.MenuItem;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.PopupMenu;
import android.widget.Toast;

import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.fragment.app.Fragment;
import androidx.navigation.Navigation;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.app.bridesassistant.DataClass.Category;
import com.app.bridesassistant.DataClass.Store;
import com.app.bridesassistant.R;
import com.app.bridesassistant.Repo.CategoriesRepo;
import com.app.bridesassistant.Repo.StoreRepo;
import com.app.bridesassistant.UI.CommonUi.Adapters.CategoriesAdapter;
import
com.app.bridesassistant.UI.CommonUi.Adapters.ViewStoresByCategroryAdapter;
import com.app.bridesassistant.Utils.Content;
import com.app.bridesassistant.Utils.Dialoge.SweetDialog;
import com.app.bridesassistant.Utils.EventBus.MessageEvent;
import com.app.bridesassistant.databinding.FragmentUserViewStoresBinding;

import org.greenrobot.eventbus.EventBus;

import java.util.ArrayList;

import cn.pedant.SweetAlert.SweetAlertDialog;

public class UserViewStores extends Fragment {
    private FragmentUserViewStoresBinding mBinding;
    private ViewStoresByCategroryAdapter categroryStoreAdapter;
    CategoriesAdapter adapter;
    ArrayList<Category> categories;
    private SweetAlertDialog loading;
    private ArrayList<Store> stores,arrayList=new ArrayList<>();
    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
```

```
        mBinding = FragmentUserViewStoresBinding.inflate(inflater,
container, false);
        stores=new ArrayList<>();
        allCategories();
        defineRecyclerStores();
        funLoading();
        clickStore();
        return mBinding.getRoot();
    }
    private void defineRecyclerStores()
    {
        categroryStoreAdapter=new
ViewStoresByCategroryAdapter(getActivity(),stores);
        mBinding.stores.setLayoutManager(new
LinearLayoutManager(getActivity()));
        mBinding.stores.setAdapter(categroryStoreAdapter);
    }


    private void allCategories() {
        categories = new ArrayList<>();
        adapter = new CategoriesAdapter(categories, getActivity());
        mBinding.allCategoriesRecycler.setLayoutManager(new
LinearLayoutManager(getActivity(), LinearLayoutManager.HORIZONTAL,
false));
        mBinding.allCategoriesRecycler.setAdapter(adapter);
        adapter.onclickItem(new CategoriesAdapter.OnItemClickListener() {
            @Override
            public void viewStores(int position) {
                Content.categoryNum=position;
                viewStoreByCategory(categories.get(position).getName());
            }
        });
        CategoriesRepo repo = new CategoriesRepo();
        repo.getAllCategories().observe(getViewLifecycleOwner(), result ->
{
            loading.dismiss();
            categories.addAll(result);
            if (result.isEmpty())
            {
                categories.add(new Category("No Categories","1"));
            }
            adapter.notifyDataSetChanged();
            getAllStores();
        });

    }

    private void getAllStores()
    {
        StoreRepo repo=new StoreRepo();
        repo.getAllStores().observe(getViewLifecycleOwner(),result->{
            loading.dismiss();
            stores.clear();
            if (result.isEmpty())
            {
```

```
                mBinding.empty2.setVisibility(View.VISIBLE);
            }
            else {
                for (Store s:result)
                {
                    if (!s.isStatus()) {
                        stores.add(s);
                    }
                }
                adapter.navigateTo(Content.categoryNum);

mBinding.allCategoriesRecycler.scrollToPosition(Content.categoryNum);

viewStoreByCategory(categories.get(Content.categoryNum).getName());
            }

        });
    }
    private void viewStoreByCategory(String category)
    {
        arrayList.clear();
        for (Store s : stores)
        {
            if (s.getType().toLowerCase().equals(category.toLowerCase()))
            {
                arrayList.add(s);
            }
        }
        if (arrayList.isEmpty())
        {
            mBinding.empty2.setVisibility(View.VISIBLE);
        }
        else{
            mBinding.empty2.setVisibility(View.GONE);
        }
        categroryStoreAdapter.updateData(arrayList);

    }
    private void clickStore()
    {
        categroryStoreAdapter.onclickItem(new
ViewStoresByCategroryAdapter.OnItemClickListener() {
            @Override
            public void viewDetails(int position) {
                Bundle b=new Bundle();
                b.putParcelable("data",arrayList.get(position));
                if (Content.type == 0)
                    Navigation.findNavController(getActivity(),
R.id.nav_host_fragment_activity_auth).navigate(R.id.action_stores_to_store
Details,b);
                else{
                    Navigation.findNavController(getActivity(),
R.id.nav_host_fragment_activity_auth).navigate(R.id.action_stores_to_store
Details2,b);
                }
            }
```

```
            @Override
            public void viewInstagram(int position) {
                openLink(arrayList.get(position).getInstaLink());
            }

            @Override
            public void viewTiktok(int position) {
                Log.e("link",arrayList.get(position).getTiktokLink());
                openLink(arrayList.get(position).getTiktokLink());
            }

            @Override
            public void viewMakeCall(int position) {
                Log.e("call",arrayList.get(position).getMobileNumber());
                call(arrayList.get(position).getMobileNumber());
            }
        });
    }
    private void openLink(String url)
    {
        Intent intent = new Intent(Intent.ACTION_VIEW);
        intent.setData(Uri.parse(url));
        startActivity(intent);
    }
    private void call(String phoneNumber) {
        if (ContextCompat.checkSelfPermission(getActivity(),
Manifest.permission.CALL_PHONE)
                != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(getActivity(), new
String[]{Manifest.permission.CALL_PHONE}, 1);
        } else {
            // Permission already granted, make the call
            Intent dialIntent = new Intent(Intent.ACTION_DIAL);
            dialIntent.setData(Uri.parse("tel:" + phoneNumber));
            if
(dialIntent.resolveActivity(getActivity().getPackageManager()) != null) {
                startActivity(dialIntent);
            } else {
                // Handle the case where no dialer is available
                Toast.makeText(getActivity(), "No dialer app found!",
Toast.LENGTH_SHORT).show();
            }
        }
    }
    private void funLoading() {
        loading = SweetDialog.loading(getContext());
        loading.show();
    }
    @Override
    public void onDestroy() {
        super.onDestroy();
        if (Content.type==0) {
            EventBus.getDefault().postSticky(new MessageEvent("2"));
        }
```

```
    }
}
```

4- category code:

```
package com.app.bridesassistant.UI.Admin.Adapters;

import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.app.bridesassistant.DataClass.Category;
import com.app.bridesassistant.R;

import java.util.ArrayList;

public class AdminCategoriesAdapter extends
RecyclerView.Adapter<AdminCategoriesAdapter.helper> {
    private OnItemClickListener mListener;
    public interface OnItemClickListener{
        void update(int pos);
        void delete(int pos);
    }
    public void setOnItemClickListener(OnItemClickListener mListener)
    {
        this.mListener=mListener;
    }
    Context context;
    ArrayList<Category> categories;
    public AdminCategoriesAdapter(Context context, ArrayList<Category>
categories) {
        this.context = context;
        this.categories=categories;
    }

    @NonNull
    @Override
    public helper onCreateViewHolder(@NonNull ViewGroup parent, int
viewType) {
        View v =
LayoutInflater.from(parent.getContext()).inflate(R.layout.recycler_admin_c
ategories, parent, false);
        return new helper(v);
    }

    @Override
    public void onBindViewHolder(@NonNull helper holder, int position) {
        holder.name.setText(categories.get(position).getName());
        holder.Update.setOnClickListener(new View.OnClickListener() {
```

```
            @Override
            public void onClick(View v) {
               mListener.update(position);
            }
        });
        holder.delete.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                mListener.delete(position);
            }
        });
    }

    @Override
    public int getItemCount() {
        return categories.size();
    }

    public class helper extends RecyclerView.ViewHolder {
        TextView name;
        Button Update,delete;
        public helper(@NonNull View itemView) {
            super(itemView);
            name=itemView.findViewById(R.id.category);

            Update=itemView.findViewById(R.id.Update);
            delete=itemView.findViewById(R.id.delete);
        }
    }
}
```

5- **Budget code:**

```
package com.app.bridesassistant.UI.User.Budget;

import android.app.AlertDialog;
import android.content.DialogInterface;
import android.os.Bundle;

import androidx.fragment.app.Fragment;
import androidx.recyclerview.widget.LinearLayoutManager;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import com.app.bridesassistant.DataClass.BudgetItemsModel;
import com.app.bridesassistant.DataClass.BudgetModel;
import com.app.bridesassistant.R;
import com.app.bridesassistant.Repo.BudgetItemsRepo;
import com.app.bridesassistant.Repo.BudgetRepo;
import com.app.bridesassistant.UI.User.Adapters.BudgetItemsAdapter;
import com.app.bridesassistant.UI.User.Adapters.RelativesAdapter;
import com.app.bridesassistant.Utils.Dialoge.SweetDialog;
import com.app.bridesassistant.databinding.FragmentUserViewBudgetBinding;
```

```
import com.github.mikephil.charting.components.Legend;
import com.github.mikephil.charting.data.PieData;
import com.github.mikephil.charting.data.PieDataSet;
import com.github.mikephil.charting.data.PieEntry;
import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.database.DatabaseReference;

import java.util.ArrayList;
import java.util.UUID;

import cn.pedant.SweetAlert.SweetAlertDialog;

public class UserViewBudgetFragment extends Fragment {
    private FragmentUserViewBudgetBinding mBinding;
    private String userId;
    private SweetAlertDialog loading;
    private BudgetRepo repo;
    private BudgetItemsRepo itemsRepo;
    private BudgetModel budgetModel;
    private BudgetItemsAdapter adapter;
    private ArrayList<BudgetItemsModel> arrayList;
    private int spentAmount = 0;

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        mBinding = FragmentUserViewBudgetBinding.inflate(inflater,
container, false);
        userId =
FirebaseAuth.getInstance().getCurrentUser().getUid().toString();
        mBinding.addTotal.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                checkEnteredData();
            }
        });
        mBinding.addSpent.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                checkEnteredData();
            }
        });
        mBinding.addButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                checkEnteredItemsData();
            }
        });
        repo = new BudgetRepo();
        itemsRepo = new BudgetItemsRepo();
        funLoading();
        defineItems();
        clickItem();
        getData();
        getDataFromFirebase();
```

```
        return mBinding.getRoot();
    }

    private void defineItems() {
        arrayList = new ArrayList<>();
        adapter = new BudgetItemsAdapter(getActivity(), arrayList);
        mBinding.items.setLayoutManager(new
LinearLayoutManager(getActivity()));
        mBinding.items.setAdapter(adapter);
    }

    private void getDataFromFirebase() {
        itemsRepo.get(userId).observe(getViewLifecycleOwner(), d -> {
            spentAmount = 0;
            arrayList.clear();
            for (BudgetItemsModel item : d) {
                spentAmount += Integer.parseInt(item.getSpent());
                arrayList.add(item);
            }
            adapter.update(arrayList);
        });
    }

    private void checkEnteredData() {
        String total = mBinding.total.getText().toString();
        String spent = mBinding.spent.getText().toString();
        if (total.isEmpty()) {
            mBinding.total.setError("Add total Amount");
        } else if (spent.isEmpty()) {
            mBinding.spent.setError("Add Spent Amount");
        } else {
            funLoading();
            BudgetModel model = new BudgetModel(Float.parseFloat(total),
Float.parseFloat(spent));
            saveData(model);
        }
    }

    private void checkEnteredItemsData() {
        String item = mBinding.item.getText().toString();
        String spent = mBinding.itemSpent.getText().toString();
        String totalSpent = mBinding.spent.getText().toString();
        int d = spentAmount + Integer.parseInt(spent);
        if (item.isEmpty()) {
            mBinding.total.setError("Add Item Name");
        } else if (spent.isEmpty()) {
            mBinding.spent.setError("Add Spent Amount");
        } else if (d > Float.parseFloat(totalSpent)) {
            funFailed("You cannot add. You have exceeded the audio. Please
increase the amount spent so that you can add.");
        } else {
            funLoading();
            BudgetItemsModel model = new BudgetItemsModel(item, spent,
UUID.randomUUID().toString());
            saveItemsData(model);
        }
```

```
    }

    private void saveData(BudgetModel model) {

        repo.saveData(model, userId).observe(getViewLifecycleOwner(),
status -> {
            loading.dismiss();
            if (status.equals("failed")) {
                funFailed("Failed to update Budget ");
            } else {
                funSuccessfully("Update budget successfully");
            }
        });
    }

    private void saveItemsData(BudgetItemsModel model) {
        itemsRepo.saveData(model, userId).observe(getViewLifecycleOwner(),
status -> {
            loading.dismiss();
            if (status.equals("failed")) {
                funFailed("Failed to Add Item");
            } else {
                funSuccessfully("Add item successfully");
                mBinding.itemSpent.setText("");
                mBinding.item.setText("");
            }
        });
    }

    private void getData() {
        repo.get(userId).observe(getViewLifecycleOwner(), result -> {
            loading.dismiss();
            budgetModel = result;
            if (budgetModel != null) {
                addDataToDesign();
            } else {
                mBinding.addSpent.setText("Add");
                mBinding.addTotal.setText("Add");
                budgetModel.setSpent(1);
                budgetModel.setTotalAmount(1);
            }
            setupPieChart();
        });
    }

    private void addDataToDesign() {
        mBinding.total.setText(budgetModel.getTotalAmount() + "");
        mBinding.spent.setText(budgetModel.getSpent() + "");
        mBinding.addSpent.setText("Update");
        mBinding.addTotal.setText("Update");

    }

    private void setupPieChart() {
        // Create data entries
        float reminder = budgetModel.getTotalAmount() -
```

```
budgetModel.getSpent();
        mBinding.spentText.setText("Spent " + budgetModel.getSpent() + "
SAR");
        mBinding.reminder.setText("reminder " + reminder + " SAR");

        ArrayList<PieEntry> entries = new ArrayList<>();
        entries.add(new PieEntry(budgetModel.getSpent(), "Spent"));
        entries.add(new PieEntry(reminder, "Remaining"));

        // Define colors (similar to the image)
        int[] colors = new int[]{
                getResources().getColor(R.color.color2),
getResources().getColor(R.color.color3)
        };

        // Create dataset
        PieDataSet pieDataSet = new PieDataSet(entries, "");
        pieDataSet.setColors(colors);
        pieDataSet.setValueTextSize(12f); // Hide values from the slices

        // Create PieData
        PieData pieData = new PieData(pieDataSet);
        mBinding.pieChart.setData(pieData);
        mBinding.pieChart.invalidate(); // Refresh chart

        // Customize Pie Chart
        mBinding.pieChart.setUsePercentValues(true);
        mBinding.pieChart.getDescription().setEnabled(false);
        mBinding.pieChart.setDrawEntryLabels(false); // Hide labels inside
slices
        mBinding.pieChart.setHoleRadius(0f); // Donut shape
        mBinding.pieChart.setTransparentCircleRadius(0f);

        // Customize legend (disable it)
        Legend legend = mBinding.pieChart.getLegend();
        legend.setEnabled(false);

        // Add animation
        mBinding.pieChart.animateY(1000);
    }

    private void clickItem() {
        adapter.setOnItemClickListener(new
BudgetItemsAdapter.OnItemClickListener() {
            @Override
            public void delete(int pos) {
                alertDialog(pos, "Delete Item", "Are you sure you need to
Delete this Item?");
            }
        });
    }

    private void alertDialog(int position, String title, String message) {
        AlertDialog.Builder builder = new
AlertDialog.Builder(getActivity());
        builder.setTitle(title)
```

```
                .setMessage(message);
        builder.setNegativeButton("No", new
DialogInterface.OnClickListener() {
            public void onClick(DialogInterface dialog, int id) {
                dialog.dismiss();
            }
        });
        builder.setPositiveButton("Yes", new
DialogInterface.OnClickListener() {
            public void onClick(DialogInterface dialog, int id) {
                deleteItem(position, title);
            }
        });
        AlertDialog alertDialog = builder.create();
        alertDialog.show();
    }

    private void deleteItem(int position, String title) {
        itemsRepo.delete(arrayList.get(position).getId(),
userId).observe(getViewLifecycleOwner(), result ->
        {
            loading.dismiss();
            if (result.equals("failed")) {
                funFailed("Failed to" + title);
            } else {
                funSuccessfully(title + " Successfully");
                getDataFromFirebase();
            }
        });
    }

    private void funLoading() {
        loading = SweetDialog.loading(getContext());
        loading.show();
    }

    private void funSuccessfully(String title) {
        SweetAlertDialog success = SweetDialog.success(getContext(),
title);
        success.show();
    }

    private void funFailed(String message) {
        SweetAlertDialog failed = SweetDialog.failed(getContext(),
message);
        failed.show();
        failed.setConfirmClickListener(new
SweetAlertDialog.OnSweetClickListener() {
            @Override
            public void onClick(SweetAlertDialog sweetAlertDialog) {
                failed.dismiss();
            }
        });
    }
}
```

CHAPTER 7: TESTING & EVALUATION

This chapter outlines the testing techniques and methodologies used to evaluate the Bride Assistant App. The goal of the testing phase is to ensure that the application functions correctly, meets user requirements, and provides a seamless experience.

Test-Case 1: User Create Account

Table 10: TC-User Create Account

Identifier	TC-User Create Account
Priority	High
Related requirements(s)	UC-User Registration
Short description	User Create Account
Pre-condition(s)	Welcome page is displayed.
Input data	Full name, Email, Password, Address, Phone number.
Detailed steps	1. The user goes to the "Create Account" section. 2. The user fills out the registration form with the required information. 3. The user submits the form.
Expected result(s)	The user is taken to a registration confirmation page.
Post-condition(s)	The user account is created, and access is granted.

This test case outlines the user registration process, detailing the typical actions taken by users and the system's responses, as shown in Table 10. It validates that new users can successfully create accounts by providing the necessary information, thereby confirming the functionality of the user onboarding process.

Test-Case 2: User Login

Table 11: TC-User Login

Identifier	TC-User Login
Priority	High
Related requirements(s)	UC-User Login
Short description	User Login
Pre-condition(s)	The user's account exists and is active.
Input data	Email, Password
Detailed steps	1. The user navigates to the "Login" section. 2. The user enters their credentials. 3. The user submits the login form.
Expected result(s)	The user is redirected to their dashboard.
Post-condition(s)	The user is logged in successfully.

This test case outlines the user login process, detailing the typical actions taken by users and the system's responses, as shown in Table 11. It validates that users can successfully log in by providing the correct credentials, thereby confirming the functionality of the user authentication process.

Test-Case 3: Budget Management

Table 12: TC-Budget Management

Identifier	TC-Budget Management
Priority	High
Related requirements(s)	UC-Manage Budget
Short description	Manage Budget
Pre-condition(s)	The user is logged in.
Input data	Budget amount, Expense details
Detailed steps	1. The user accesses the budget management section. 2. The user sets a budget and enters any expenses. 3. The user saves the updated budget.
Expected result(s)	The budget is updated and displays the new totals.
Post-condition(s)	The user's budget is tracked correctly.

This test case outlines the budget management process, detailing the typical actions taken by users and the system's responses, as shown in Table 12. It validates that users can successfully manage their budget by setting a budget amount and entering expenses, thereby confirming the functionality of the budget management system.

Test-Case 4: Vendor Directory Access

Table 13: TC-Vendor Directory Access

Identifier	TC-Vendor Directory Access
Priority	High
Related requirements(s)	UC-Vendor Directory
Short description	Access Vendor Directory
Pre-condition(s)	The user is logged in.
Input data	Search criteria (e.g., vendor type)
Detailed steps	1. The user goes to the vendor directory. 2. The user choose a category. 3. The user views the list of vendors.
Expected result(s)	A list of vendors matching the search criteria is displayed.
Post-condition(s)	The vendors are accessible for contact.

This test case outlines the process for accessing the vendor directory, detailing the typical actions taken by users and the system's responses, as shown in Table 13. It validates that users can successfully access the vendor directory by applying search criteria, thereby confirming the functionality of the vendor directory system.

Test-Case 5: Task Organization

Table 14: TC-Task Organization

Identifier	TC-Task Organization
Priority	High
Related requirements(s)	UC-Task Organization
Short description	Organize Tasks
Pre-condition(s)	The user is logged in.
Input data	New task details
Detailed steps	1. The user navigates to the task management section. 2. The user creates a new task and sets a deadline. 3. The user saves the task.
Expected result(s)	The task is added to the user's list.
Post-condition(s)	The task is saved and tracked properly.

This test case outlines the process for organizing tasks, detailing the typical actions taken by users and the system's responses, as shown in Table 14. It validates that users can successfully create and organize tasks by entering new task details, thereby confirming the functionality of the task management system.

Test-Case 6: Guest List Management

Table 15: TC-Guest List Management

Identifier	TC-Guest List Management
Priority	High
Related requirements(s)	UC-Guest List Management
Short description	Manage Guest List
Pre-condition(s)	The user is logged in.
Input data	Guest name, Relationship.
Detailed steps	1. The user goes to the guest list management section. 2. The user adds a new guest and their relationship. 3. The user saves the guest list.
Expected result(s)	The guest list is updated with new entries.
Post-condition(s)	The guest list is managed correctly.

This test case outlines the process for managing the guest list, detailing the typical actions taken by users and the system's responses, as shown in Table 15. It validates that users can successfully manage the guest list by adding guest names and their RSVP statuses, thereby confirming the functionality of the guest list management system.

Test-Case 7: Add Wedding Information

Table 16: TC-Add Wedding Information

Identifier	TC-Add Wedding Information
Priority	High
Related requirements(s)	UC-User Login
Short description	User can add wedding date and place.
Pre-condition(s)	User is logged in and on the wedding information page.
Input data	Wedding date and place.
Detailed steps	1. The user navigates to the "**Wedding Information**" screen. 2. The the user enters a new wedding date in the provided field. 3. The the user enters a new wedding place in the provided field. 4. The user clicks the "**Update**" button to save the information..
Expected result(s)	• The system should save the updated wedding date and place, displaying a success message. • The remaining days until marriage should automatically update based on the new date.
Post-condition(s)	The wedding information is updated in the system, reflecting the new date and place.

This test case validates that users can effectively add their wedding date and place, confirming the functionality of the wedding information management system, as shown in Table 16.

Test-Case 8: Admin Manage Vendor Store

Table 17: TC-Admin Manage Vendor Store

Identifier	TC-Admin Manage Vendor Store
Priority	High
Related requirements(s)	UC-Vendor Directory
Short description	Admin Manage Vendor Store
Pre-condition(s)	Admin is logged in and on the Stores page.
Input data	Vendor selection (Update, Block), Category(Add or Delete).
Detailed steps	1. The admin navigates to the "Stores" section. 2. The admin selects a vendor from the list. 3. The admin chooses an action: • If "**Update**" is selected, the admin enters new vendor details and confirms the changes. • If "**Block**" is selected, the admin confirms the action to block the vendor. 4. For the new category: • If **adding** a category, the admin enters the new category name and confirms the addition. • If **deleting** a category, the admin selects the category to delete and confirms the action.
Expected result(s)	1. If "**Update**" is selected: The system should save the updated vendor details and display a success message. 2. If "**Block**" is selected: The vendor should be marked as blocked, and the system should confirm the action with a success message. 3. If **adding** a new category: The new category should be assigned to the vendor, and a success message should be displayed. 4. If **deleting** a category: The category should be removed from the vendor, and a confirmation message should be displayed.
Post-condition(s)	The vendor details are updated in the system, the vendor is blocked from public view, a new category is added, or a category is deleted.

This test case outlines the process for managing vendor stores, detailing the typical actions taken by the admin and the system's responses, as shown in Table 17. It validates that the admin can successfully update vendor details, block vendors, and manage categories, confirming the functionality of the vendor management system.

Test-Case 9: Forgot Password

Table 18: TC-Forgot Password

Identifier	TC-Forgot Password
Priority	High
Related requirements(s)	UC-User Login
Short description	User can reset their password.
Pre-condition(s)	User is on the login page.
Input data	Email address for password reset.
Detailed steps	1. The user clicks on the "**Forgot Password**?" link. 2. The user enters their email address in the provided field. 3. The user clicks the "**Send**" button to request a password reset.
Expected result(s)	The system should send a password reset link to the provided email address and display a success message.
Post-condition(s)	The user receives an email with instructions to reset their password.

This test case outlines the process for resetting a password, detailing the actions taken by the user and the system's responses, as shown in Table 18. It validates that users can successfully request a password reset, confirming the functionality of the password recovery system.

Test-Case 10: User Logout

Table 19: TC-User Logout

Identifier	TC-User Logout
Priority	High
Related requirements(s)	UC-User Login
Short description	User can log out of the application.
Pre-condition(s)	User is logged in.
Input data	None
Detailed steps	The user clicks on the "**Logout**" option.
Expected result(s)	• The user should be logged out of the application and redirected to the login page. • The session should be terminated, and the user should no longer have access to protected resources.
Post-condition(s)	The user is logged out, and their session is cleared.

This test case validates that users can log out effectively, confirming the functionality of the logout process, as shown in Table 19.

Test-Case 11: Add Comments and Rating

Table 20: TC-Add Comments and Rating

Identifier	TC-Add Comments and Rating
Priority	High
Related requirements(s)	UC-User Login
Short description	User can add comments and ratings for stores.
Pre-condition(s)	User is logged in and on the stores page.
Input data	Selected store, Comment text , Rating value (e.g., 5 stars)
Detailed steps	1. The user navigates to the "**Stores**" section. 2. The user selects a store. 3. The user clicks the exclamation mark button to open the comment and rating interface. 4. The user enters a comment in the provided text field. 5. The user selects a rating (e.g., 5 stars) using the rating bar. 6. The user clicks the "Send" button to save the comment and rating.
Expected result(s)	• The system should save the comment and rating, displaying a success message. • The new rating should be reflected in the store's average rating. • The comment should be visible in the store's review section.
Post-condition(s)	The store's ratings and comments are updated in the system.

This test case ensures that users can submit comments and ratings, confirming the proper functionality of the comment and rating system, as shown in Table 20.

Test-Case 12: Access Social Media and Contact Links

Table 21: TC-Social Media and Contact

Identifier	TC- Access Store Social Media and Contact Links
Priority	Medium
Related requirements(s)	UC-User Login
Short description	User can access store social media accounts and contact numbers.
Pre-condition(s)	User is logged in and on the stores page.
Input data	Selected store.
Detailed steps	1. The user navigates to the "**Stores**" section. 2. The user selects a store. 3. The user can clicks the Instagram link, TikTok link or store's contact number.
Expected result(s)	• The Instagram or TikTok link should redirect to the store's Instagram account. • Clicking the contact number should initiate a call to the store.
Post-condition(s)	The user successfully accesses the store's social media accounts and can call the store.

This test case verifies that users can access store social media accounts and contact information, ensuring a smooth user experience when connecting with stores, as shown in Table 21.

CHAPTER 8: CONCLUSION AND FUTURE WORK

10.1 Conclusion

The **Bride Assistant App** is a feature-rich smartphone application created especially to make wedding preparations easier for brides in Saudi Arabia's Eastern Province. By combining all necessary tools into a single, intuitive Android platform, it tackles the emotional, practical, and cultural difficulties that are frequently faced during wedding planning. Although there are numerous wedding applications available, very few are specifically designed for the Saudi market, especially with features that link directly to regional vendors and customs. This project's cultural significance is what makes it significant. To lessen stress and improve the planning process, the app has countdown elements, task organization, vendor directories, budgeting tools, and guest list management.

The project's uniqueness is found in its cultural sensitivity and comprehensive approach. In contrast to international apps, the Bride Assistant App respects and incorporates regional wedding traditions while offering direct access to Saudi-based suppliers and services. It stands out for its smooth design and variety of functions, which enable brides to oversee almost every element of their wedding from a single platform, providing a digital solution that empowers and reduces stress. The app has an instant impact since it helps brides stay organized, manage their spending, and make wise decisions in a user-friendly, culturally appropriate layout.

With its current support for essential features like work management, budget tracking, and vendor engagement, the app is a potent tool for contemporary brides. It is created utilizing the Waterfall technique to guarantee a systematic and thoroughly documented development process, and it makes use of Firebase for secure login and real-time data management. Admin tools are also included for effective vendor and user management.

10.2 Future Work

Looking ahead, there are multiple opportunities to enhance the application's overall value and extend its reach, such as:

- Improve the app's suggestions by recommending vendors that match the user's preferences and budget.
- Offer augmented reality previews of venues and decorations for a more immersive experience.
- Expand the list of vendors to cover more cities and regions in Saudi Arabia.
- Add different language options so more people can use the app easily.
- Collaborate with local wedding businesses to provide special deals and allow users to book services directly through the app.

REFERENCES

1. AbuOdeh, M., & Atabeh, L. (2023). Dream Wedding. In *Software Graduation Project*. https://repository.najah.edu/server/api/core/bitstreams/96ba1770-30d6-4f94-82a4-ac01148c0c51/content

2. Sangchan, C. (n.d.). *Wedding Event Planning and Management: A Case Study of The Hug Wedding Planner and Decor* (By Thammasat University, Internship Advisor, Assistant Professor Anyarin Amornissariyachai, Mrs. Chatda Chalermkiat, & The Hug Wedding Planner and Decor). https://digital.library.tu.ac.th/tu_dc/digital/api/DownloadDigitalFile/dowload/193605

3. Sawalmeh, R., & Barakat, R. (2023). *The Big Day: Wedding planner* (By AN-NAJAH NATIONAL UNIVERSITY). https://repository.najah.edu/bitstreams/121dcd52-6e96-481b-8c5c-940fb2d5ed5a/download

4. Senarath, U. S. & Rajarata University of Sri Lanka. (2021). *Waterfall Methodology, Prototyping, and Agile Development*. https://doi.org/10.13140/RG.2.2.17918.72001

5. Durgekar, S. R., Rahman, S. A., Naik, S. R., Kanchan, S. S., & Srinivasan, G. (2024b). A Review Paper on Design and Experience of Mobile Applications. *ICST Transactions on Scalable Information Systems*. https://doi.org/10.4108/eetsis.4959

6. *Registry App - Free Honeymoon Registry by Honeyfund, the #1 Cash Wedding Registry. No fees, no strings.* (n.d.). Honeyfund. https://www.honeyfund.com/registry-app

7. *Mila Celebration - Your Saviour on Special Occasion*. (2024, October 7). Mila Celebrations. https://mila.day/en/home/

8. App Store. (2017, July 10). *Wedding Planner* . https://apps.apple.com/us/app/wedding-planner/id1236966921

9. *The Big Day - Wedding Planning App*. (n.d.). https://thebigday.app/

10. Singh, R. K., Singh, P., & Bathla, G. (2020). User-Review Oriented Social Recommender System for Event Planning. *Ingénierie Des Systèmes D Information*, *25*(5), 669–675. https://doi.org/10.18280/isi.250514

11. *Saudi Weddings: A Traditional Affair*. (2018, March 26). About Her. https://www.abouther.com/node/9126/people/features/saudi-weddings-traditional-affair
12. *Honeydue | Finance App for Couples*. (n.d.). Honeydue. https://www.honeydue.com/

13. weddinghappy.com. (2022, November 1). *WeddingHappy: The wedding app that gets you down the aisle*. weddinghappy.com |. https://www.weddinghappy.com/

14. *Todoist | A To-Do List to Organize Your Work & Life*. (n.d.). https://todoist.com/

15. Wulaningratri, G. P., Wardhana, M. I., & Hermanto, Y. a. L. (2021). Design Elements on Wedding Planner Platforms as the Fundamental of User Interface Design. *Advances in Social*

Science, Education and Humanities Research/Advances in Social Science, Education and Humanities Research. https://doi.org/10.2991/assehr.k.211212.022

16. (*Wedding Planning for Dummies*, n.d.)

17. Mariana. (2024, May 13). *Embracing the Evolution of Weddings*. The Wedding Academy. https://weddingacademyglobal.com/2024/06/embracing-the-evolution-of-weddings/

18. Jahroh, M. R. W. S. I. F. S. (2020). Factors That Influence Consumer Decision in Wedding Vendor Selection. *Jurnal Manajemen*, *24*(3), 392. https://doi.org/10.24912/jm.v24i3.675

19. *Saudi Arabia Wedding Services Market 2024-2032 | Size,Share, Growth*. (n.d.). MarkWide Research. https://markwideresearch.com/saudi-arabia-wedding-services-market/

20. App Store. (2019b, July 22). *Save The Date Rsvp Your Event*. App Store. https://apps.apple.com/sa/app/save-the-date-rsvp-your-event/id1464412490

21. Lamothe, M., Guéhéneuc, Y., & Shang, W. (2021). A Systematic Review of API Evolution Literature. *ACM Computing Surveys*, *54*(8), 1–36. https://doi.org/10.1145/3470133

22. Thirunahari, S. & School of Computing, Electrical, and Applied Technology, Unitec Institute of Technology, Auckland, New Zealand. (n.d.). A Development Framework for Software Integration Projects – Case Study: Web App Integration with OpenWeather API. In *A Development Framework for Software Integration Projects*. https://www.researchbank.ac.nz/server/api/core/bitstreams/9be50cb2-42da-4e53-8739-1b60933af693/content

23. Alwin, E. (2024). *Using ChatGPT to create mobile apps : Analysis of using ChatGPT as a programming tool for developing smaller mobile applications*. DIVA. https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A1865437&dswid=6775

24. Zhao, X., & Li, X. (2024). The Latest Trends and Challenges in Enterprise Application Development with Java. *Advances in Computer Signals and Systems*, *8*(2). https://doi.org/10.23977/acss.2024.080207

25. G, T. C., & Devi, J. (2021). *A Study and Overview of the Mobile App Development Industry*. https://supublication.com/index.php/ijaeml/article/view/424
26. Pierce, M., & Jiang, P. (2021). User Acceptance of Mobile Event Apps: A Cross-Cultural Study. International Journal of Internet Marketing and Advertising, 15(2), 113-131. https://www.inderscienceonline.com/doi/abs/10.1504/IJIMA.2021.112787

27. García Revilla, R., Ozturk, C., & Einsle, T. (2023). Advances in event management using new technologies and mobile applications. International Journal of Event and Festival Management, 14(2), 103-118. https://www.researchgate.net/publication/366837934_Advances_in_event_management_using_new_technologies_and_mobile_applications

28. Vyas, B. (2023). Security Challenges and Solutions in Java Application Development. ResearchGate. https://www.researchgate.net/profile/Bhuman-Vyas/publication/376717198_Security_Challenges_and_Solutions_in_Java_Application_Development/links/6584c7250bb2c7472bfe4564/Security-Challenges-and-Solutions-in-Java-Application-Development.pdf

29. Ribeiro, A., Ferreira, J. F., & Mendes, A. (2021). A framework for intelligent wedding planning with user preferences and constraints. https://ieeexplore.ieee.org/abstract/document/9724955/

30. Gaikwad, A. (2021). Firebase - Overview and Usage. [ResearchGate]. https://www.researchgate.net/profile/Anil-Gaikwad-12/publication/362539877_FIREBASE_-OVERVIEW_AND_USAGE/links/62efc738505511283e9a5318/FIREBASE-OVERVIEW-AND-USAGE.pdf

APPENDIX: Glossary

- **API:** Application Programming Interface, a set of tools and protocols for building software applications.
- **Budget Tracking:** A feature in the app that helps users manage and monitor their expenses.
- **Waterfall Methodology:** A sequential development approach where each phase depends on the deliverables of the previous phase.
- **Vendor:** A service provider who offers wedding-related services such as catering, photography, or decoration.
- **FAB:** Stands For The Floating Action Button and It's typically used for a primary, positive action on a screen.